

UAM SGFinDex OAuth Proxy Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAM SGFinDex OAuth Proxy Implementation Guide.....	3
2. Introduction to UAM SGFinDex OAuth Proxy Implementation.....	4
2.1. SGFinDex and AccessMatrix UAM Integration.....	5
2.2. Implementation Planning.....	15
3. UAM SGFinDex OAuth Proxy Configuration.....	19
3.1. Configure Admin Console.....	21
3.2. Configure SGFinDex OAuth Proxy.....	27
4. UAM SGFinDex OAuth Proxy Deployment.....	40
5. Securing API Gateway Access to SGFindex Supporting APIs with OAuth 2.0.....	41
6. UAM SGFinDex OAuth Proxy APIs and Endpoints.....	45
6.1. SGFinDex OAuth Proxy Endpoints.....	46
6.2. SGFinDex Supporting APIs.....	66
7. Appendices.....	80

1. Preface - UAM SGFinDex OAuth Proxy Implementation Guide

This document provides the concept, implementation steps and API integration details of the Singapore Financial Data Exchange (SGFinDex) with the AccessMatrix Universal Access Management (UAM) system.

Intended Reader

It is intended as a reference for the security administrators, i-Sprint's Professional Service personnel, and developers to carry out the following operations:

- Deployment of OAuth Proxy SGFinDex
- Configuration of OAuth Proxy SGFinDex
- API gateway integration
- Configuration of SGFinDex endpoints

Document Scope

This document provides the necessary steps to configure SGFinDex endpoints with details on its concept and API implementation.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [AccessMatrix Common Deployment Guide](#)

2. Introduction to UAM SGFinDex OAuth Proxy Implementation

This document provides instructions on the deployment, integration and maintenance of the SGFinDex OAuth Proxy.

Terminology

This table lists some common terms that are used in this document. It is not intended as a comprehensive definition for each item.

Term	Description
Financial Information Service Provider (FISP)	It represents the service provider insurer.
API Gateway	It serves as the API proxy to forward the requests to the OIDC endpoints or FISP resource endpoints.
Service Consumer	The financial planning application that requests the end user's policy data.
Gateway Client	The OAuth Client of the FISP's API Gateway.
API gateway token	It refers to the access token of the OAuth Client of the FISP's API Gateway that will be used as the bearer token for calling SGFinDex supporting APIs.

About SGFinDex

Singapore Financial Data Exchange (SGFinDex) is a digital infrastructure that uses a national digital identity and centrally manages an online consent system to enable individuals to access, through applications, their financial information held across different government agencies and financial institutions.

SGFinDex is designed with data protection and privacy controls in place for personal information. Financial data can only be retrieved with the explicit consent of the individual through Singpass, and individuals retain control over which participating entities they wish to share their data with. An individual's consent to link participating entities to SGFinDex lasts for only one year from the time the user's account is linked to SGFinDex. Their consent needs to be given again after a year.

2.1. SGFinDex and AccessMatrix UAM Integration

SGFinDex is a protocol for financial data exchange, built on top of the OAuth 2.0 authorization framework.

AccessMatrix UAM system supports OpenID Connect 1.0 with the OAuth Proxy as the OAuth endpoints component. To integrate with SGFinDex, an **SGFinDex OAuth Proxy** component is provided as an enhanced component to conform the OAuth Proxy to the protocol of the SGFinDex. This component is also designed with a thorough flow that enables the FISP's API gateway to securely retrieve the user's consented policy data from its resource endpoint to serve its service consumer. For this, some supporting API endpoints are also available in this component to cater for specific use cases, such as request signature and token verification, and policy data signing and encryption endpoints.

The SGFinDex OAuth Proxy has the following:

- **SGFinDex OAuth Endpoints**

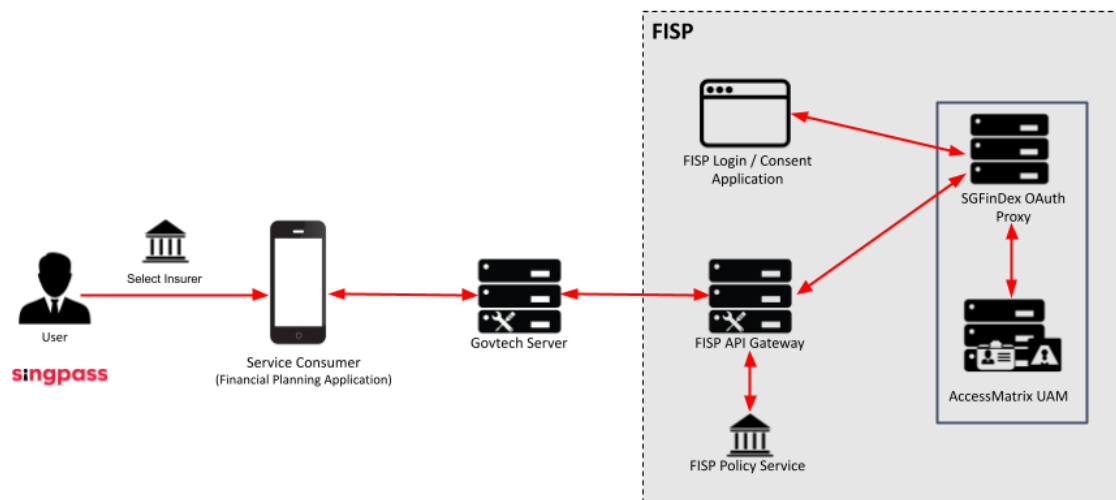
These provide authorization, token, and revocation endpoints to enable the authorization and linking of SGFinDex and FISP account.

- **SGFinDex Supporting APIs**

These provide REST APIs to support the FISP API Gateway in providing resource endpoints (e.g. /getpolicy, ...). For this role, the APIs can be used to verify the SGFinDex payload, verify the access token, and to sign and encrypt the resource endpoint response.

SGFinDex and AccessMatrix UAM Component Diagram

Below is the diagram that illustrates how the AccessMatrix UAM and the SGFinDex OAuth Proxy integrate with SGFinDex:



AccessMatrix UAM and the SGFinDex OAuth Proxy act as the FISP Identity Provider. AccessMatrix components are not internet facing and rely on the API Gateway (or Reverse Proxy component) to forward the request to the intended endpoint(s).

As shown in the diagram, FISP's Login/ Consent Application is not part of the AccessMatrix Product. Please refer to [this](#) section to ensure smooth integration with the SGFinDex OAuth Proxy.

In the figure above:

1. End user accesses the Financial Planning application and logs in with Singpass authentication.
2. End user selects the insurer from which they intend to pull their insurance policy details to be consolidated with the Financial Planning application.
3. GovTech Server receives the request from the Financial Planning application and triggers an OAuth Request to the FISP Identity Provider to authenticate and prompt the end user for consent.
4. FISP API Gateway forwards the OAuth request to SGFinDex OAuth Proxy and that prompts the user to first perform FISP login, then grant consent to GovTech/the Financial Planning application. These steps facilitate authorization and linking between SGFinDex's client and its user's FISP account.

-
5. If this process is successful, the Financial Planning application, which is also the service consumer, obtains an access token to retrieve its end user's policy details from the FISP Policy Service.
 6. FISP API Gateway receives request from the Service Consumer to retrieve end user's policy details. It shall call the supporting APIs provided by SGFinDex OAuth Proxy for the following services:
 - Verify its service consumer's request.
 - Sign and encrypt its end user policy details before returning the response result to the caller.
 7. The encrypted policy details of the end user are decrypted by the Financial Planning application, and then further used and displayed for the end user's financial planning.

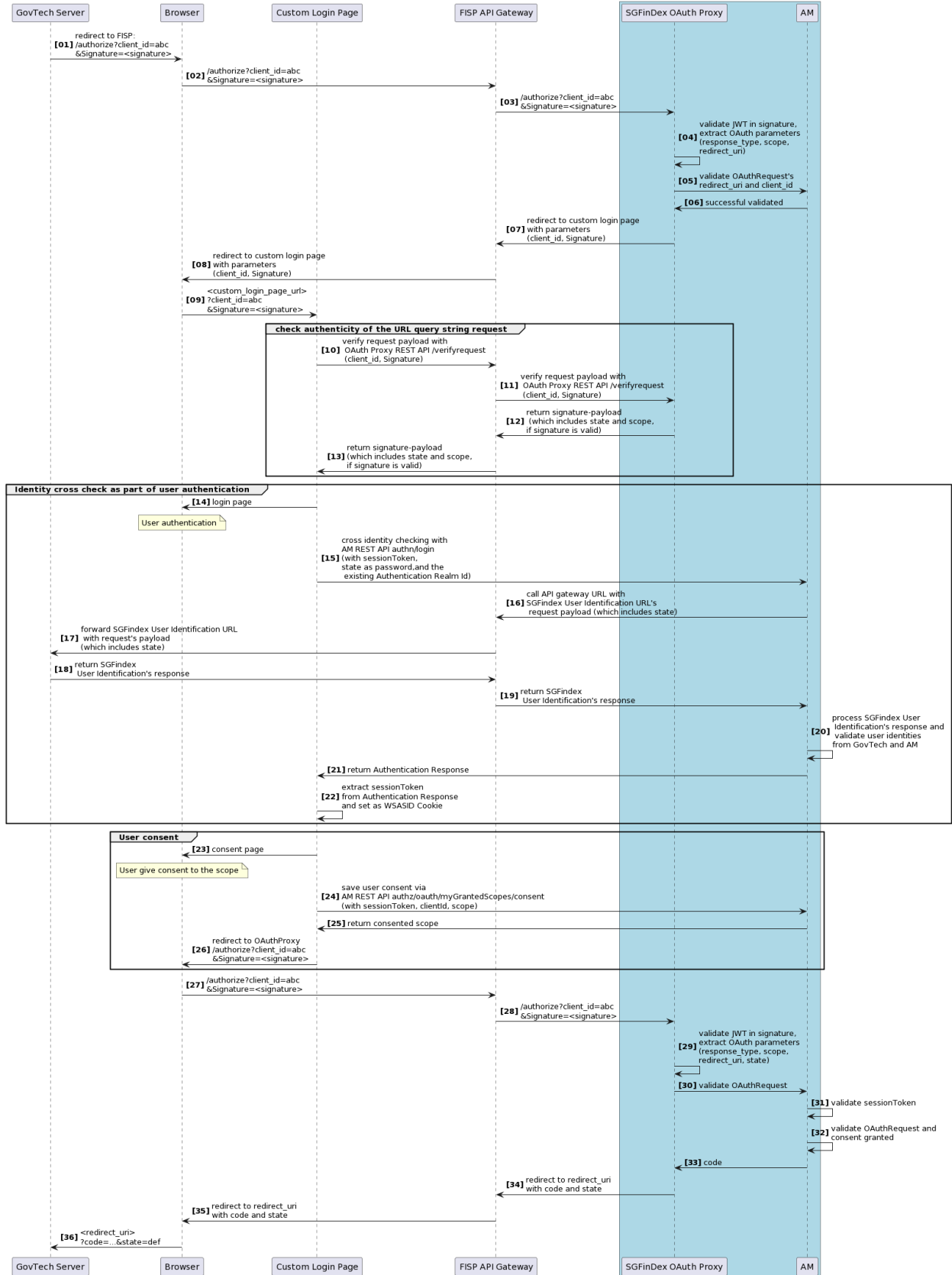
SGFinDex OAuth Endpoints

SGFinDex OAuth endpoints are the endpoints of the FISP IDP. These endpoints are configured in the SGFinDex OAuth Proxy WAR file.

Authz Use Case

The Authorization Code Request endpoint is used by the GovTech Server to request the authorization code, which is exchanged for an access token and a refresh token.

Authorization Endpoint API



In this diagram, it is assumed that the end user has logged in to a Financial Planning application with Singpass and triggered a request to allow the GovTech Server to retrieve the insurance policy details, from the selected insurer, on the user's behalf.

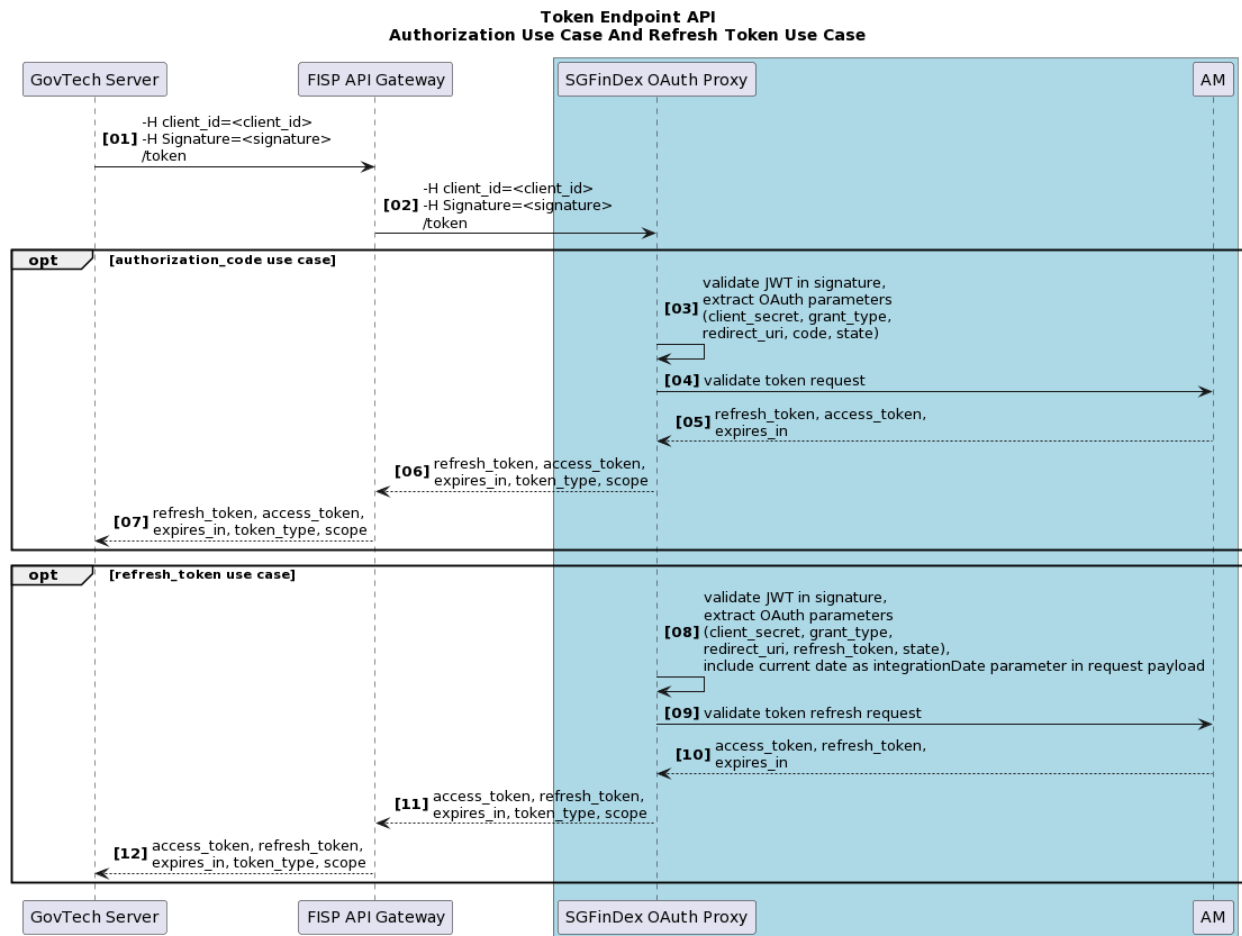
- **Step 1-3:** GovTech Server translates the request by redirecting the browser to the FISP API Gateway, which passes the request via the proxy GET Method to the SGFinDex OAuth Proxy's /authorize endpoint.
 - **Step 4-6:** The proxy validates the JWT signature and extracts the OAuth parameters in the JWT to make the request to AM in order to validate the request itself.
 - **Step 7-9:** Since the user is not yet authenticated, SGFinDex OAuth Proxy redirects them to the Custom Login Page to login.
 - **step 10-13:** Custom Login Page is required to call the SGFinDex OAuth Proxy supporting API - /verifyrequest - to validate the authenticity of the incoming request where the signature and validity of the request are checked. If the request is deemed valid, the response payload which includes the state (for identity cross-checking) and the scope (for the consent page to prompt the user for consent) is returned.
 - **Step 14-15:** If the user is successfully authenticated, the Custom Login Page does [identity cross-checking](#) by calling the AM5 REST API via authn/login. The authentication realm used in the API must contain the SGFinDex Identity Login Module as the last login module.
 - **Step 16:** AM calls API gateway URL with the SGFinDex user identification URL's request payload, which includes state for user identification.
 - **Step 17:** The API Gateway triggers the actual request of the SGFinDex user identification URL - with the given request payload - to the GovTech Server for the actual processing.
 - **Step 18-19:** GovTech Server returns the response of the call to AM via the API Gateway.
 - **Step 20-21:** AM processes the SGFinDex user identification response and validates the user identity obtained from GovTech Server and AM5 user repository. If successfully validated, the Custom Login Page receives an authentication response with the Session Token.
 - **Step 22:** The Custom Login Page extracts the Session Token from the authentication response and sets it as the WSASID cookie.
 - **Step 23:** This is followed by a consent prompt for the user to grant consent to the GovTech Server to retrieve their policy details on their behalf.
 - **Step 24:** Once the user grants consent, the Custom Login Page calls the AM5 REST API to save the user consent via `authz/oauth/myGrantedScopes/consent`.
-

-
- **Step 25:** The Custom Login Page receives the consented scopes as the response.
 - **Step 26-33:** The Custom Login Page redirects the user to the SGFinDex OAuth Proxy's Authorize endpoint again. This time, as the user has been authenticated, the OAuth authorization is validated and processed.
 - **Step 34-36:** For a successful use case, the authorization code is returned from AM, passing through the proxy to the API gateway, and then redirected back to the GovTech Server.

Get Token Use Case

Get Token endpoint:

- **Authorization Code use case**
It is used to exchange an authorization code for an access token and a refresh token.
 - **Refresh Token use case**
It is used to exchange a refresh token for a new access token and refresh token.
 - **Client Credential use case (FISP internal use)**
It is used by the API Gateway to obtain an access token so that it can authenticate itself to the SGFinDex supporting APIs. Please refer to [this](#) section for more details on this use case.
-

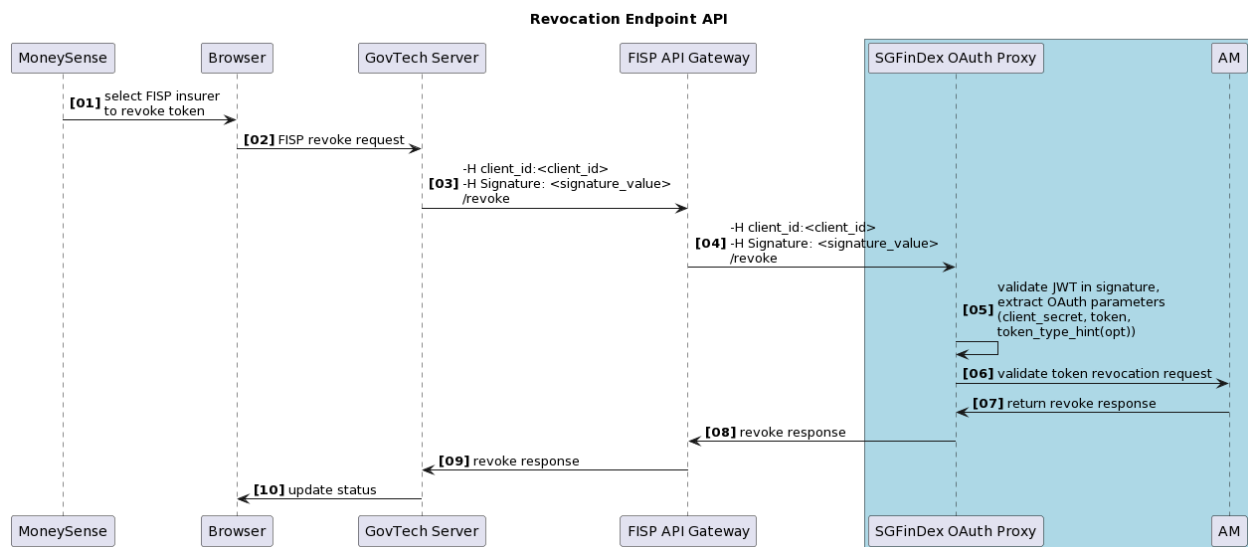


In this diagram, the GovTech Server makes a request to the Token endpoint, either to exchange an authorization code for an access token, or to exchange a refresh token for a new access token and refresh token.

The GovTech Server constructs and sends the request to the FISP API Gateway, which passes it on to the SGFinDex OAuth Proxy. The proxy validates the JWT signature and extracts the OAuth parameters in the JWT to make the request to AM in order to exchange the code or token for the new token. The new tokens are returned back with some additional details through the proxy and gateway to the caller.

In the refresh token use case, where a refresh token is exchanged for a new access token and refresh token, the new refresh token's lifetime will be calculated starting from current date (the date of the exchange).

Revoke Token Use Case



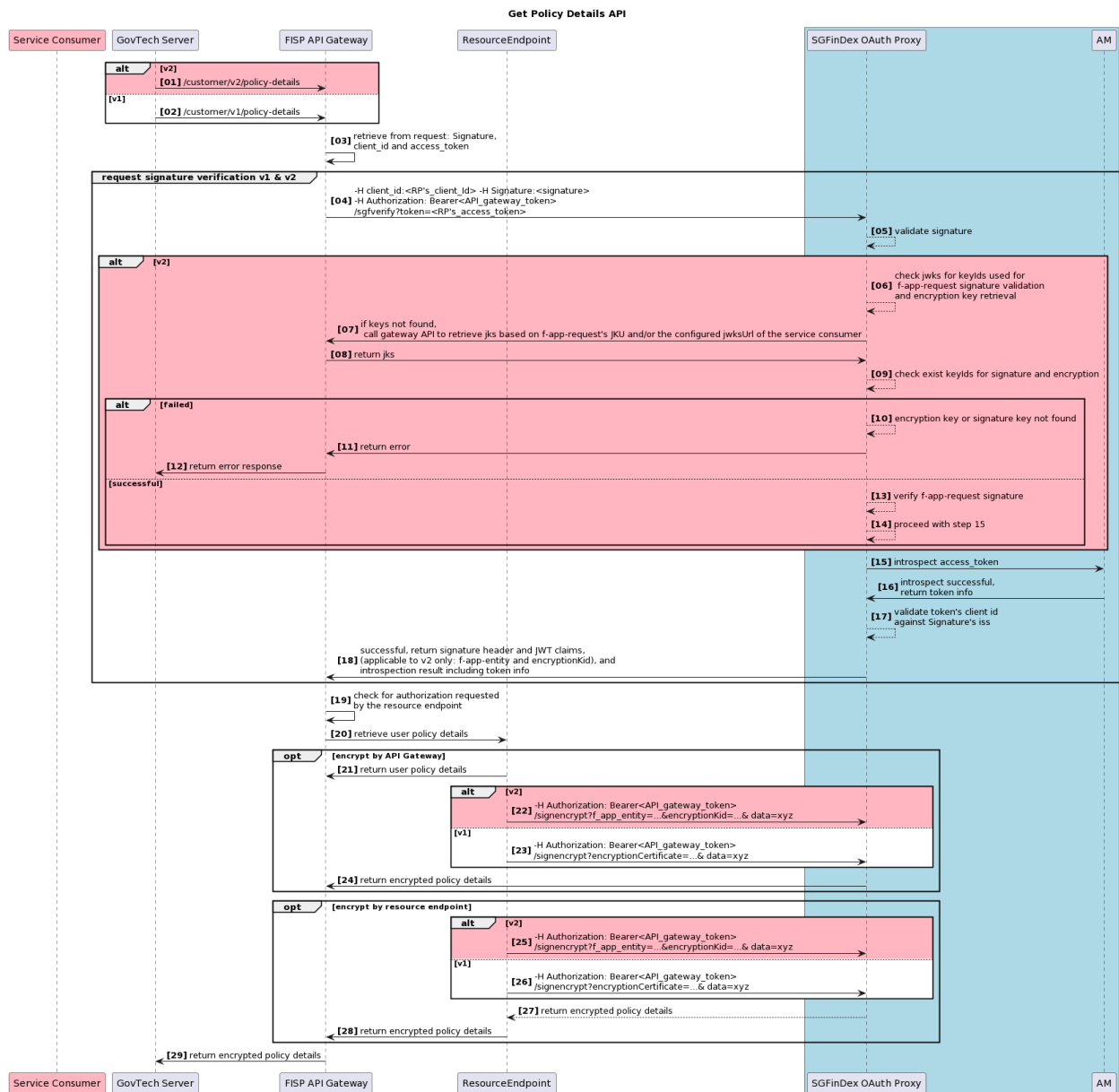
In this diagram, MoneySense sends a request to revoke a token through the browser. The request is passed to the GovTech Server, which makes the API call to the FISP API Gateway. The request is then passed to the SGFinDex OAuth Proxy for validation. The proxy then extracts the OAuth parameters from the validated JWT to make the token revocation request to AM directly. The revocation response is returned through the proxy and gateway to GovTech Server, who then updates the request status to the browser.

SGFinDex Supporting APIs

The Get Policy Details API endpoint is an endpoint for the GovTech Server to request policy details from the FISP.

According to the SGFinDex specifications, the FISP IDP of the Get Policy Details API needs to verify the caller's request to ensure the caller is authorized before retrieving the policy details on behalf of the end user. Subsequently, the API signs and encrypts the policy details of the end user with the encryption certificate provided in the request.

In the context of deployment, the FISP API Gateway plays the role of the Get Policy Details API to validate access token validity status and consent of end user with the IDP of FISP before returning the encrypted policy details to the caller.



Below are the steps of the diagram that illustrate the flow of the Get Policy Details API, which makes use of two SGFinDex supporting APIs called [/sgfverify](#) and [/signencrypt](#). These two APIs are for internal use and should not be internet facing.

- **Step 1-2:** It is assumed that the end user has logged in to access the financial planning application, and that the application has triggered a request to the GovTech Server to retrieve the end user insurance policy details. GovTech Server makes a request to the FISP to retrieve the user policy details, and the request is intercepted by FISP API Gateway.
- **Step 3:** FISP API Gateway retrieves Signature and bearer token from the request.
- **Step 4-18:** To check the authorization and validity of the token, FISP API Gateway calls the SGFinDex OAuth Proxy to validate the signature and introspect the token. This request validates

the integrity of the JWT requests (Signature, and optionally f-app-request signature too). The signature JWT claims and token info are returned and used by the API gateway to authorize the next step.

- **Step 19:** The API Gateway checks for authorization requested by the resource endpoint based on the response result (e.g. scope, user identity and etc.).
- **Step 20:** If the caller is authorized, the gateway then requests the user policy details from the resource endpoint. There are two possible implementations for the next step.
- **Step 21-24:** This solution has the policy details encrypted by the API Gateway. It calls the SGFindex OAuth Proxy to sign encrypt the policy details.
- **Step 25-28:** This solution has the policy encrypted by the resource endpoint. It calls the SGFindex OAuth Proxy to sign encrypt the policy details.
- **Step 29:** FISP API Gateway should receive the response - the encrypted policy detail or the error response - and return the response result to the GovTech Server.

Verify Signature Token Introspection API

- This API is called to verify the JWT signature and introspect the RP access token, returning the OAuth claims in the JWT as well as the token info.
- It requires the caller to present the access token obtained from the client credential grant as the bearer token. Read [this](#) to find out more.
- You may click [here](#) to find out more on the API specification.

Sign and Encrypt API

- This API signs the policy details and encrypts the result using the passed-in encryption certificate.
 - It requires the caller to present the access token obtained from the client credential grant as the bearer token. Read [this](#) to find out more.
 - You may click [here](#) to find out more on the API Spec.
-

2.2. Implementation Planning

Implementation Checklist

1. **AM5 Admin Console and SGFinDex OAuth Proxy Configuration**
 - a. [UAM SGFinDex OAuth Proxy Configuration](#)
2. **Custom Login Application Integration**
 - a. [Verifying Authenticity of Request Signature](#)
 - b. [Identity Cross-checking](#)
 - c. [Setting Authentication Flow Browser Cookie](#)
 - d. [Saving Consent](#)
3. **API Gateway Preparation**
 - a. [Preparation to Use Gateway Token to Call Supporting API](#)
 - b. Perform authorization checking with scope defined by [Configure OAuth Authorization Server](#) before calling the sign and encrypt supporting API

Implementation Planning

Security Consideration

Some of the endpoints in the SGFinDex OAuth Proxy are meant for internal use only (not directly internet-facing), such as for the SGFinDex supporting APIs. For better security, it is best to have a reverse proxy or the API gateway to filter the incoming requests before calling the SGFinDex OAuth Proxy. Although server-to-server authentication is in place to be able to call the SGFinDex supporting APIs, it is considered a best practice to ensure that these APIs are accessible only to the API gateway or resource server (depending on the implementation).

SGFinDex OAuth Proxy requires a keystore to be configured to store the signing key. It is best practice to limit the OS file permissions to be readable only by the OS user used to run the SGFinDex OAuth Proxy instance. The keystore also requires a keystore password to be configured in SGFinDex OAuthProxy configuration file. This can be encrypted using AccessMatrix's Encryptor Utility Tool.

Integration Consideration

Verifying Authenticity of Request Signature

The role of the custom login page is to prompt the user for authentication and ask the user to grant consent. Prior to that, it should verify the authenticity of the request before proceeding. The SGFinDex OAuth Proxy provides an endpoint, namely [/verifyrequest](#), to allow the custom login page to check the request signature of the incoming request. If the signature is validated (i.e. its signature is verified), then it can trust that the incoming request was triggered from the legitimate client by prompting user for further authentication. In the event that the request is not valid (respMessage is not successful), it should not proceed with the request.

Identity Cross-checking

After user authentication is completed, there is a need to cross-check the AM5 user identity with the GovTech user identity (Singpass identity) from the infra/GovTech server.

This part of the validation is handled by the SGFinDex identity login module. Therefore, this login module needs to be [set as the last login module](#) when the [authentication realm is configured](#).

This identity cross-checking process is done by the custom login page - via the AM5 REST API `authn/login` - by passing in the session token, user Id, state as password, and realm Id (authentication realm Id) that was configured with the SGFinDex identity login module.



Note: `authn/login` REST API parameter values:

- **session token**
 - If the previous user authentication was done using AM5, the session token is extracted from the authentication response and used as the `sessionToken` parameter of the `authn/login` REST API for identity cross-checking.
 - If the previous user authentication is not done using AM5, an empty value is used as the `sessionToken` parameter of the `authn/login` REST API for identity cross-checking.
- **state**
 - The state is passed in as the `password` parameter. This value can be obtained from the response of [/verifyrequest](#).

Please refer to `authn/login` of AccessMatrix REST API for more details regarding the error response of this API.

AM is not internet facing, and the SGFinDex identity login module typically calls the API Gateway to forward the SGFinDex user identification URL to the actual API URL of GovTech.

For your reference, the following is the payload of the SGFinDex user identification POST request:

Request Payload	
JSON	<pre>{ "state": <state>, "client_assertion": <client assertion JWT containing claims sub, jti, aud, iss, iat, exp>, "client_assertion_type": <format of assertion defined by authz server> }</pre>

The client assertion JWT is used to authenticate the client. It is signed with the private key of the Service Provider and contains the following claims:

- sub: The client_id issued by Myinfo upon onboarding.
- jti: The JWT ID, a random unique number.
- aud: The URL that partner application is calling.
- iss: The client_id issued by Myinfo upon onboarding.
- iat: The timestamp at which the JWT was created and issued.
- exp: The expiry timestamp, capped at a maximum of 300 seconds (5 minutes).

The client assertion type specifies the format of the assertion, as defined by the authorization server.

Authentication Flow Browser Cookie

For deployment, the Custom Login Application must share the same domain as SGFinDex OAuth Proxy to facilitate cookie reading by the SGFinDex OAuth Proxy.

- FISP login application needs to set WSASID cookie for the following attributes accordingly:

Cookie Attribute	Value
Cookie name	WSASID

Cookie Attribute	Value
HTTP-Only	true
Secure	true
max-age	-1
SameSite	same as SGFindex OAuth proxy setting

- Other settings like cookie path and cookie domain should match the [configuration](#) in SGFindex OAuth Proxy, or else you need to ensure it is still accessible by SGFindex OAuth Proxy.

Saving of User's Consent

Once user authentication is completed, user would be prompt to grant consent. After granting authorization or consent for the scope(s), the saving of the consent has to be done by calling AM5 REST API using user's session token together with the OAuth client Id and the granted scope(s). Please refer to [authz/oauth/myGrantedScopes/consent](#) in the [AccessMatrix REST API Specification](#) for the more details.

To view scopes granted to an end user, please refer to:

- [authz/oauth/myGrantedScopes/get](#)
 - This API is used to get end user's consented scopes based on his or her login session (aka session token).
- [authz/oauth/grantedScopes/get](#)
 - This API is used to retrieve end user's consented scopes by user identity when admin/agent session is used.

Performance Consideration

Most of the expensive processing is done by the SGFindex OAuth Proxy, including the verification of request signature, payload signing of the access token, and signing/ encryption of the policy details.

3. UAM SGFinDex OAuth Proxy Configuration

Setup Steps

Prior to SGFinDex OAuth Proxy configuration, it is assumed that the AccessMatrix UAM server has been set up properly. Otherwise, the administrator should proceed to set up the AccessMatrix UAM server first. Refer to [AccessMatrix Common Deployment Guide](#).

Having ensured that the UAM server have been installed properly, the administrator may proceed to perform the following configuration steps:

Configuration in Admin Console

- [Upload OAuth License](#)
- [Create OAuth Client for Service Consumer](#)
- [Configure OAuth Authz Server](#)
- [Create OAuth Client for API Gateway](#)

Configuration in SGFinDex OAuth Proxy

- [Configure Proxy to Connect to the AccessMatrix Server](#)
 - [Configure Login Page Redirect URL](#)
 - [Configure Session Token Cookie Domain and Path](#)
 - [Configure Error Page URL \(Optional\)](#)
 - [Configure OpenID Provider/Resource Signing Key](#)
 - [Configure SGFinDex OAuth Proxy Introspect Endpoint URL](#)
 - [Configure X.509 Certificate Path for GovTech Client](#)
 - [Configure Whitelisted CNs of Service Consumers](#)
 - [Configure OAuth Client ID of API Gateway](#)
 - [Configure Interval Between Housekeeping of Expired Cached Gateway Token\(s\) \(Optional\)](#)
 - [Configure SGFinDex Proxy Log Properties \(Optional\)](#)
 - [Configure Connection Pool Settings to AccessMatrix Server \(Optional\)](#)
-

Upload OAuth License

Before doing any configuration in the Admin Console, secure the server license required to implement AccessMatrix UAM as OIDC. Ensure that the **UAM FEATURE** is enabled, and a value for the **OAuth Client** license feature is specified.

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
2. Select the Server Id and click **Edit**.
3. Select the **Licensed Features** tab and select the first radio button. Browse for the license file.
4. Click **Upload Licence File** to upload the license file to the server.
5. Save the changes.
6. Restart the AccessMatrix server for the new license to take effect.

3.1. Configure Admin Console

The following sections contain the configurations required in Admin Console to integrate SGFinDex OAuth Proxy with AccessMatrix UAM system.

Create OAuth Client for GovTech Server

Perform the steps below to create the OAuth client for the GovTech Server:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click the **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select the **Confidential** option as the client type and click **Next >**.
3. Select the **Server Side Web App** option as the application type and click **Next >**.
4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
*OAuth Client Id	Unique identifier of OAuth client, e.g. "finApp".
*OAuth Client Name	Name of OAuth client, e.g. "finApp".
Description	Description of OAuth client.

5. Fill in the basic setting information:

Attribute Name	Description
Grant Type	Value is "authorization_code". (uneditable)
Token endpoint authentication method	Value is "ClientSecret". (uneditable)
Response Type	Value is "code". (uneditable)
Use OpenID Connect	This attribute is checked by default. (uneditable)
*OAuth redirect URIs	Specifies the URI that the end users will be redirected to, along with the authorization code returned by the OAuth authorization server. (mandatory)
Consent rule	Indicates if the OAuth client can be trusted or not. Options:

Attribute Name	Description
	○ "TRUSTED" ○ "NOT_TRUSTED"
*Value to return as Subject (sub) value	Indicate where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token) for this OAuth client. This setting will be used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. (Mandatory) Options: ○ "Id" ○ "Name" ○ "DN" ○ "Email" ○ "Secondary Account" ○ "User Response"
User Response to return as Subject (sub) value	This attribute is only required if the Value to return as Subject (sub) attribute is configured with "User Response". This allows other alternatives to retrieve a user identity (using user response's predefined field/attributes, or by customization of user response).

- Click the **Finish** button. If there's no problem, the **Create OAuth Client** page will be populated with the information configured in **OAuth Client Configuration Wizard**.
- Set the value of **Access Token Format** to "JWT".
- Set the value of **Return Refresh Token** to "true".
- Set the value of **Refresh Token Lifetime** to "8760" hours (1 year).
- Set the value of **Use Refresh Token Integration Date** to "true".
- Set the value of **Consent rule** to "NOT_TRUSTED".
- Set the value of **Revoke Consented Scopes When Revoking Token** to "Revoke All".
- Use the **Value to return as Subject (sub) value**, and optionally **User Response to return as Subject (sub) value**, setting(s) to clearly specify the mapping of the user identity in AM5. This value is important as it is the user identity value that would be returned by the [/sgfverify](#) supporting API for the API gateway to check for the authorization for calling the Get Policy Details API. Please refer to step 9 of the [use case](#) illustration to understand how this value would be possibly used.
- Save the changes.



Note: Refer to *UAM OAuth and OIDC Implementation Guide - Configure Authorization Code Flow for Server-side (Web) Application* for the list of attributes and its descriptions.

Configure OAuth Authz Server

There is a need to configure the OAuth Authz Server to set the validity time of the authorization code and scope to allow the user to grant authorization to the OAuth Client for accessing their data. Scope values can be used in the application to specify what access privileges are being requested for access tokens. To configure:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Authz Server**.
2. Click **Edit** on the **View OAuth Authz Server** page.
3. Click the **JWT/OIDC Tokens** tab and configure the **Authorization Code Validity Time (in seconds)** attribute value to "30".
4. Following steps is to define the scope(s) required in the bearer token for calling the FISP Get Policy Details API.
 - Click the **Scope Management** tab.
 - Click the **Edit** button in the **Scopes** attribute value to launch the **Scopes Configuration** dialog.
 - Click the **Add** button to add a custom scope.
 - Specify the **Scope Name**.
 - Leave the **User Response as Scope Value** field as empty as it is used to return the resource or user info for the granted scope for OIDC use case.
 - Select the **Consent Rule** value. Set the field to "REQUIRED".
 - You may click the **Add** button if you want to add additional custom scopes. Otherwise, click **OK** to add the custom scope to list of **Scopes**.



Note: The scope(s) defined here should be used and further elaborated in the consent UI. End users would be prompted with this UI after user authentication with FISP IDP, and this is to grant authorization or consent to the GovTech server to retrieve policy details on behalf of the end-user.

5. Save the changes.

Configure ID Token Signing Key

Signing key for OAuth client application can be system-generated or configured in key store. It is highly recommended to use system-generated signing key instead of key store signing key as key store signing key is meant for legacy application, and it does not support key rotation.

Perform the following steps to configure the system-generated signing key:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Authz Server**.
2. Click the **Edit** button and fill-in the following attributes under the **System generated key pair for signing in ID Token (JWT)** header:
 - **RSA key length** - Options: "2048" or "4096". The default value is 2048.
 - **Wrapping Key** - A wrapping key is required to generate the OpenID Connect signing key pair. This key is used to protect system generated signing key pair.
 - **No. of public keys to remember for verification** - The value should not be less than 2. The default value is 2.
3. Save the changes.



Note:

- To generate the first signing key, click the **Generate OpenID Connect Signing Key Pair** button. The **Key generation for ID Token (JWT) signing** dialog will be displayed. Click **OK** to confirm the generation of new key.
- To rotate signing key, use the system-generated signing key pair instead of key store signing key pair. Click the **Generate OpenID Connect Signing Key Pair** button.

Create OAuth Client for FISP's API Gateway

Follow the steps provided [here](#) to create the gateway client in Admin Console.

Create Login Module for Identity Cross-Checking

Perform the steps below to create the login module for identity cross-checking:

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
-

-
2. Click **Create** and select the "SGFinDex Identity Login Module" implementation from the **Choose Login Module Implementation** page.
 3. Enter a **Login Module Id** (e.g. "SGFinDexLogin") and a **Login Module Name** (e.g. "SGFinDex Login Module").
 4. In the **Configuration** tab, under the **Login Configuration** header, configure the following attributes:
 - a. Select the **Login Policy** and the **Authentication Level** values, if necessary.
 5. Under the **Identity Checking** header, configure the following attributes:
 - a. Select the appropriate **Environment** value.
 - b. (Optional) Specify the **SGFinDex User Identification URL** to be used if it is not one of the default URLs specified in the **Environment** attribute description.
 - c. Specify the **SGFinDex APP ID** that was supplied upon the SGFinDex onboarding.
 6. Under the **Request Signing Key Store** header, configure the following attributes:
 - a. Specify the **Keystore** file path, **Keystore Passphrase**, and **Key Alias** values for the FISP insurer's signing keystore.

 **Note:** The provided keystore must be in the "JKS" file format. During the generation of the key pair, please ensure that "JKS" keystore type is used.

- b. Specify the **Active Key Id**. This configuration step differs based on the SGFinDex API version you are using:

(Mandatory) Specify the key Id from the published JWKS URL of the SGFinDex identity login module keystore. To obtain this value, perform the following steps:

- i. Configure the SGFinDex identity login module keystore settings in the **sgf.yaml** file. Refer to the [Signing Keys for SGFinDex Identity Login Module](#) section.
- ii. Once the SGFinDex OAuth Proxy has been deployed and the AccessMatrix server is running, obtain the key Id by accessing the [SGFinDex OAuth Proxy's JWKS endpoint](#).

V2



Notes:

- The SGFinDex OAuth Proxy's JWKS endpoint returns both the FISP OP signing keys and the SGFinDex identity login module signing keys. Ensure you retrieve the key Id that corresponds to the SGFinDex identity login module keystore.
- The key pair corresponding to the key Id must be an EC (Elliptic Curve) key pair. Ensure the key type is "EC".

V1	(Optional) Specify the key Id to be used as the FISP OP public key Id, if needed. This setting can be used if there is a requirement to use a specific value as the key Id instead of the key Id generated by AM.
----	---

7. In the **Identity Mapping** tab, specify the **User's Custom Attribute Name** value so that the module knows where to obtain AM5 user identity for identity cross-checking.
8. Specify and configure the attributes in the **HTTP Proxy** tab. The proxy settings are required, as AM5 is not internet facing and requires a proxy to forward API requests to the GovTech server.
9. Save the changes.

Refer to [AccessMatrix Common Administration Guide - Configure Login Module](#) for detailed information about login modules.

Create Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Click **Create**.
3. Enter an **Authentication Realm Id** (e.g. "SGFinDexRealm") and an **Authentication Realm Name** (e.g. "SGFinDex Realm").
4. Specify the **User Lookup Module** value that will be used to lookup AM5 user, e.g. "SystemLookup", "AttributeBasedLookup", etc.
5. Specify the **Login Module** used for identity cross-checking (e.g. "SGFindex Identity Login Module") that was created in the previous section.



Note: This login module must be specified as the last login module value if multiple-factor authentication is involved. For example, if 2 login modules are required, the identity cross-checking login module should be specified as the value of the **Second(2nd) Login Module**.

6. Save the changes.

3.2. Configure SGFinDex OAuth Proxy

The following sections contain the configurations required in SGFinDex OAuth Proxy to integrate with AccessMatrix UAM system.

Configure Proxy to Connect to the AccessMatrix Server

The configuration below establishes the connection between SGFinDex OAuth Proxy and AccessMatrix server. To configure:

1. Navigate to `<SGF_HOME>/WEB-INF/` and open **web.xml**.
2. Look for the `restAPIURL` element and modify the value to refer to the relevant AccessMatrix server as shown below:

XML
<pre><context-param> <param-name>restAPIURL</param-name> <param- value>http://localhost:8080/am5/rest</param-value> </context-param></pre>

To support client-authenticated SSL (a mandatory configuration for production environments), the following settings should be configured:

- Modify the `restAPIURL` value to use a secure connection to the AccessMatrix server as shown below:

XML
<pre><context-param> <param-name>restAPIURL</param-name> <param- value>https://localhost:8443/am5/rest</param-value> </context-param></pre>

- Configure the following client keystore properties:

XML
<pre><context-param> <param-name>am.hc4.ssl.keystore</param-name> <param-value>/opt/acmatrix/tomcat/conf/clp- client.keystore</param-value> </context-param></pre>

```
<context-param>
  <param-name>am.hc4.ssl.keystore-passphrase</param-name>
  <param-
value>AesEncryptor:pWtZZ0N6bfIvSlBBF0zXC0ZwS1Kny/GexGVFpaqWI1Q
=</param-value>
</context-param>
```



Note: Refer to [AccessMatrix Common Login Page Deployment Guide - List of Configurable Parameters](#) for the description of the properties.

3. Save the changes.
4. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure Login Page Redirect URL

1. Navigate to `<SGF_HOME>/WEB-INF/` and open **web.xml**.
2. Look for the loginRedirectURL element as shown below:

XML

```
<filter>
  <filter-name>SGFAuthFilter</filter-name>
  <filter-
class>com.isprint.oauth.proxy.sgf.filter.SGFOAuthRequestServletFilt
er</filter-class>
  <init-param>
    <param-name>loginRedirectURL</param-name>
    <param-value>/oauth2login</param-value>
  </init-param>
</filter>
...
<servlet>
  <servlet-name>OAuthRequestServlet</servlet-name>
  <servlet-
class>com.isprint.oauth.proxy.OAuthRequestServlet</servlet-class>
  <init-param>
    <param-name>loginRedirectURL</param-name>
    <param-value>/oauth2login</param-value>
  </init-param>
  ...
</servlet>
```

3. Modify the existing value, which is `/oauth2login`, to the actual login redirect URI value of the corresponding custom login page. Some examples include:
 - a. `https://<SGFindex_domain>/`
 - b. `https://<SGFindex_domain>/sgf/clp`

-
- c. `https://<SGFindex_domain>/?appSource=sgfindex`



Note: Examples

like `https://<SGFindex_domain>/?` and `https://<SGFindex_domain>/?appSource=sgfindex&` are invalid.

4. Save the changes.
5. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure Session Token Cookie Domain and Path

Upon FISP login, an AccessMatrix session token cookie (WSASID cookie) is generated. This session cookie is used to verify user identity and the session's validity; and this can be set to a domain and a context path.

To configure:

1. Navigate to `<SGF_HOME>/WEB-INF/` and open **web.xml**.
2. Look for the cookie-related elements as shown below:

XML

```
<context-param>
  <param-name>cookiePath</param-name>
  <param-value>/</param-value>
</context-param>
<context-param>
  <param-name>cookieDomain</param-name>
  <param-value></param-value>
</context-param>
<context-param>
  <param-name>cookieSameSite</param-name>
  <param-value></param-value>
</context-param>
```

3. Specify the domain value to allow the session cookie be set in the specified domain. By default, domain value is empty, thus setting the session cookie in the default domain.



Note: You are required to specify the domain value if you are using Internet Explorer.

4. Specify the context path value. By default, path is set to `/`, thus allowing the session cookie to be set in all context paths in the current domain.
 5. Specify the `cookieSameSite` value if required. By default, `cookieSameSite` is empty, thus it would depend on browser's default behavior.
-



Note: The purpose of `cookieSameSite` is to avoid CSRF (Cross-Site Request Forgery).

6. Save the changes made.
7. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure Error Page URL

The SGFinDex OAuth Proxy WAR file contains an error page that is used by the FISP's IDP to forward its user to an error page in the event of a failed customer registration request from GovTech to the FISP insurer. During the customer registration request process, the SGFinDex OAuth Proxy Authorization Code Request endpoint is triggered to facilitate the FISP IDP's user authentication and consent. Below is the listing of the possible errors that cause the endpoint to forward the browser to the error page:

- Missing or invalid redirect URI
- Missing or invalid client
- Missing or invalid request signature

sgferror.jsp is a simple error page in SGFinDex OAuth Proxy war that displays an error reference Id for troubleshooting use. You may customize and modify this error page to redirect users to the external error page of the FISP insurer.

Below is the configuration of the error page URL should there be any changes on the error page URL:

1. Navigate to `<SGF_HOME>/WEB-INF/` and open **web.xml**.
2. Look for the `errorPageUrl` sections `SGFAuthFilter` and `OAuthRequestServlet`, as shown below:

XML

```
<filter>
<filter-name>SGFAuthFilter</filter-name>
<filter-
class>com.isprint.oauth.proxy.sgf.filter.SGFOAuthRequestServletFilt
er</filter-class>
...
<init-param>
    <param-name>errorPageUrl</param-name>
    <param-value>/sgferror.jsp</param-value>
</init-param>
</filter>
<servlet>
    <servlet-name>OAuthRequestServlet</servlet-name>
    <servlet-
```

```
class>com.isprint.oauth.proxy.OAuthRequestServlet</servlet-class>
...
<init-param>
  <param-name>errorPageUrl</param-name>
  <param-value>/sgferror.jsp</param-value>
</init-param>
</servlet>
```

3. Change the existing `/sgferror.jsp` param-value accordingly.



Note: Your error page still has to be a local file. It cannot be an external URL.

4. Save the changes made.
5. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure OpenID Provider/Resource Signing Keys

There are two sets of signing key pairs to be configured in the **sgf.yml** file:

Signing Keys

This key pair is used to sign:

- Access token JWT when the Get Token endpoint is called
- Policy details data when the Sign and Encrypt endpoint is called
- Request of the SGFinDex user identification URL is called by the SGFinDex identity login module

The public key of this key pair is published in the [JWK URL](#) of the SGFindex OAuth Proxy.

Perform the following steps to configure the keys used for signing the JWT payload:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Look for the `signingKeys` key and configure the following keystore settings, as shown below:

YAML

```
# Financial Information Service Provider Configuration
fisp:
...
signingKeys:
  keystore: /opt/acmatrix/tomcat/conf/fisp-signing.jks
  keystorePassphrase:
Aes256Encryptor:SJc3Kj2584am6sSWqG17jdEjqLxXHK9+r...7xWdQdUEEVx7e
  keyAlias: signfsip
```

The value of each setting should match the value of the corresponding keystore-related attribute configured in the [Create Login Module for Identity Cross-Checking](#) section.

- `keystore` - The Java keystore file that contains the key used for signing the JWT payload.

Note: The provided keystore must be in the "JKS" file format. During the generation of the key pair, please ensure that "JKS" keystore type is used.

- `keystorePassphrase` - The password of the Java keystore. This password or passphrase can be a clear password or an encrypted password value.

Note: Refer to the [Common Deployment Guide - Encryptor Utility Tool](#) section for the encryption steps.

- `keyAlias` - The key alias of the keystore.

3. Save the changes.

4. Restart the SGFinDex OAuth Proxy for changes to take effect.

Signing Keys for SGFinDex Identity Login Module

This key pair is used to sign the client assertion JWT that is used by the SGFinDex Identity login module for user identity cross-checking. The public key of this key pair is published in the [JWK URL](#) of the SGFinDex OAuth Proxy.

Before proceeding, ensure that the key Id contained in the JWKS is a static value based on the key pair. The key pair must also be an EC (Elliptic Curve) key pair. You can verify this by examining the JWKS - the value of the `key` field should be "EC".

After verifying the above, perform the following steps to configure the keys used for signing the JWT payload:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Look for the `signingKeysSGFIdentityModule` key and configure the following keystore settings, as shown below:

YAML

```
# Financial Information Service Provider Configuration
fisp:
  ...
  signingKeysSGFIdentityModule:
```



```
keystore: /opt/acmatrix/tomcat/conf/SGFIM-signing.jks
keystorePassphrase:
Aes256Encryptor: SJc3Kj2584am6sSWqG17jdEjqLxXHK9+r...7xWdQdUEEVx7e
keyAlias: signfsip
```

Configure the following values:

- **keystore** - The Java keystore file that contains the key used for signing the JWT payload.

i **Note:** The provided keystore must be in the "JKS" file format. During the generation of the key pair, please ensure that "JKS" keystore type is used.

- **keystorePassphrase** - The password of the Java keystore. This password or passphrase can be a clear password or an encrypted password value.

i **Note:** Refer to the [Common Deployment Guide - Encryptor Utility Tool](#) section for the encryption steps.

- **keyAlias** - The key alias of the keystore.

3. Save the changes.

4. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure SGFindex OAuth Proxy Introspect Endpoint URL

The SGFindex OAuth Proxy Introspect endpoint URL is used by the `/sgfverify` endpoint when calling the internal endpoint for token introspection.

Perform the following steps to configure the SGFindex OAuth Proxy Introspect endpoint URL:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Look for the `sgfindexOAuthProxyIntrospectURL` key. Replace the existing value with the appropriate URL. For example:

YAML

```
# Financial Information Service Provider Configuration
fisp:
...
# SGFindex OAuth Proxy URL Introspect URL
e.g. "https://SGFProxy.myportal.com.sg:8443/introspect"
sgfindexOAuthProxyIntrospectURL:
https://testserver:8443/oauth-proxy-sgf/introspect
session:
...
```

-
3. Save the changes.
 4. Restart the SGFindex OAuth Proxy for the changes to take effect.

Configure Authentication Max Age (Optional)

The authentication max age dictates how long a session token is considered valid after the latest login. Should a session token that is considered as invalid be used, the user will be redirected to the configured login page for reauthentication.

Perform the following steps to configure the authentication max age:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Look for the session key and configure the `authnMaxAge`:

YAML

```
# Financial Information Service Provider Configuration
fisp:
  ...
  signingKeys:
  ...
  session:
    # validity period of last authentication in minutes,
    value range is 2 - 5.
    authnMaxAge: 2
```

- o `authnMaxAge` - The maximum amount of time, in minutes, past the latest login for the session token to still be considered valid.

3. Save the changes.
4. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure X.509 Certificate Path for GovTech Client

Perform the following steps to configure the X.509 Certificate Path:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Ensure the `jwtksUrl` key is disabled (i.e. it is commented out).
3. Look for the `x509CertPEMPATH` key and specify the value, as well as the value for `clientId` as shown below:

YAML

```
# Singapore Financial Data Exchange Configuration
govtech:
  clientId: gov1
  #jwksUrl: http://www.123.com/jwks.json
  x509CertPEMPath: C:\sgf.pem
```

- `clientId` - Refers to the OAuth `client_id` that was auto-generated (e.g. ZU1iWicoF66HJ3nO2gXbng) when FISP registered GovTech as the OAuth client in AccessMatrix UAM.
 - `x509CertPEMPath` - Path of X.509 Certificate in PEM format. It is used to obtain public key from the OAuth client in order to verify the Signature payload obtained from the OAuth requests.
4. Save the changes.
 5. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure JWKS URL for GovTech Client (optional)

Perform the following steps to configure the JSON Web Key Set (JWKS) URL:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **`sgf.yml`**.
2. Ensure the `x509CertPEMPath` key is disabled (i.e. it is commented out).
3. Look for the `jwksUrl` key and specify the value, as well as the value for `clientId` as shown below:

YAML

```
# Singapore Financial Data Exchange Configuration
govtech:
  clientId: gov1
  jwksUrl: http://localhost:8080/am5/wpe/jwks.json
  #x509CertPEMPath: C:\sgf.pem
```

- `clientId` - Refers to the OAuth `client_id` that was auto-generated (e.g. ZU1iWicoF66HJ3nO2gXbng) when FISP registered GovTech as the OAuth client in AccessMatrix UAM.
 - `jwksUrl` - URL of the OAuth client's JWKS endpoint. It is used to obtain public key from the OAuth client in order to verify the Signature payload obtained from the OAuth requests.
4. Save the changes.
 5. Restart the SGFinDex OAuth Proxy for changes to take effect.
-

Configure Whitelisted CNs of Service Consumers

FISP needs to configure a whitelist of the fully qualified common names of each service consumer as it will be used for certificate validation.

Perform the following steps to configure the whitelisted common names (CNs):

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Look for the `whitelistedCNs` key and specify the list of acceptable CNs as shown below:

```
YAML

# Singapore Financial Data Exchange Configuration
govtech:
  ...
  serviceConsumers:
    encryptionKeys:
      whitelistedCNs:
        - www.service.consumer.1.com
        - www.service.consumer.2.com
```

- `whitelistedCNs` - refers to the list of allowed common names that are used to verify the encryption certificate of the service consumer.
3. Save the changes.
 4. Restart the SGFinDex OAuth Proxy for changes to take effect.

i **Note:** If `*` is used as part of the CN, you need to use `"`. For example, `"*.consumer.1.com"`.

Configure JWKS URL of Service Consumers

As of SGFinDex V2, Service Consumers are identified via an `f_app` entity value assigned by the Central Infra. This value, along with the corresponding JWKS URL of the Service Consumer, should be configured in the **sgf.yml** file.

Perform the following steps to do so:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Look for `serviceConsumerConfig` and specify the values for `f_app_Entity` and `jwksURL` as shown below:

```
YAML
```

```
serviceConsumerConfig:
  #- f_App_Entity: id1
  #   jwksUrl: http://example1/jwks.json
  #- f_App_Entity: id2
  #   jwksUrl: http://example2/jwks.json
```

- **f_app_Entity** - the name assigned to the Financial Institution (the Service Consumer) by the Central Infra.
- **jwksURL** - the URL pointing to the **jwks.json** of the Service Consumer, which contains the key Ids used for the f-app request signature validation and encryption key retrieval.



Note: If any of the configured JWKS URLs fail to return the jwks.json, an exception will be logged and the process will proceed with the retrieval of the jwks.json files from the other jwks URL(s).

3. Save the changes.
4. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure Gateway Client ID

Follow the steps provided [here](#) to configure the allowed Gateway Client in SGFinDex OAuth Proxy.

Configure Interval Between Housekeeping of Expired Cached Gateway Token(s) (Optional)

The gateway token is used by the API gateway to authenticate to supporting APIs as these APIs are called. These tokens are cached in SGFinDex OAuth Proxy to improve the performance of the /signencrypt supporting API. These cached token(s) may turn invalid due to token expiry and would be purged by a background job based on certain interval configured here.

To modify the interval between the housekeeping jobs:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Look for the **gatewaytokenHousekeepInterval** key and specify the value as shown below:

YAML

```
supportingAPIsAuthentication:
  ...
  gatewaytokenHousekeepInterval: 60
```

-
- gatewaytokenHousekeepInterval - refers to the interval between housekeeping of the expired cached gateway token in minutes. The minimum value of the interval is 10 minutes.
3. Save the changes.
 4. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure SGFinDex Proxy Log Properties (Optional)

SGFinDex OAuth Proxy uses Apache Log4j 2 as the logging utility. It defines the log level and log output file location.

To configure:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **log4j2.properties**.
2. Modify the log level, if necessary.
3. Save the changes.
4. Restart the SGFinDex OAuth Proxy for changes to take effect..

Configure Connection Pool Settings to AccessMatrix Server (Optional)

Connection pooling refers to the method of creating a pool of connections and caching those connections so that it can be reused again. It improves the response time of a web-based application. This setting is used to fine tune the connection to AccessMatrix server.

To configure:

1. Navigate to `<SGF_HOME>/WEB-INF/` and open **web.xml**.
2. Set the connection timeout value of SGFinDex OAuth Proxy to AccessMatrix server by configuring the following parameter values:

XML

```
<context-param>
  <param-name>am.hc4.timeout.conn</param-name>
  <param-value>20000</param-value>
</context-param>
<context-param>
  <param-name>am.hc4.timeout.socket</param-name>
  <param-value>30000</param-value>
```

```
</context-param>
<context-param>
  <param-name>am.hc4.pool.maxconn_per_route</param-
name>
  <param-value>50</param-value>
</context-param>
```

3. You may add more connection pool parameters by referring to [AccessMatrix Common Login Page Deployment Guide - List of Configurable Parameters](#) section.
 4. Save the changes.
 5. Restart the SGFinDex OAuth Proxy for changes to take effect.
-

4. UAM SGFinDex OAuth Proxy Deployment

SGFinDex OAuth Proxy can be deployed by installing the Web Archive (WAR) package. To deploy:

1. Secure the SGFinDex OAuth Proxy WAR file from the AccessMatrix CD-Image. The filename format is **AccessMatrix-x.x.xxxx-GA-oauth-proxy-sgf.war**, e.g. **AccessMatrix-5.7.2.7206-GA-oauth-proxy-sgf.war**.
2. (Optional) For easier reference, rename the WAR file to **oauth-proxy-sgf.war**.
3. The SGFinDex OAuth Proxy is a stateless web service, which means you can deploy the WAR file (**oauth-proxy-sgf.war**) to any web application container supported by AccessMatrix core products version 5.7.2 and up (i.e. Apache Tomcat and JBoss Web Server).
4. If using Tomcat, create an *oauth-proxy-sgf* folder under */opt/acmatrix/tomcat/webapps/*.
5. Extract the contents of **oauth-proxy-sgf.war** to this folder.

i **Note:** In this guide, <SGF_HOME> is used to refer to the installation directory of SGFinDex OAuth Proxy components, e.g. */opt/acmatrix/tomcat/webapps/oauth-proxy-sgf/*.

SGFinDex OAuth Proxy Components

Below is the list of the folders and sub-folders inside the OAuth Proxy SGF WAR file:

Folder/File Name	Sub-folder/File Name	Description
WEB-INF	classes\log4j2.properties	This file contains the configurations of log level and log output file location. Refer to the Configure SGFinDex Proxy Log Properties section for configuration details.
	classes\sgf.yml	This file contains the FISP and SGFinDex configurations.
	lib	The libraries used for the SGFinDex OAuth Proxy.
	web.xml	This file contains the configurations of the AccessMatrix REST API URL, cookie settings, connection pool configuration, and filter mapping URL pattern for the SGFinDex filter.
sgferror.jsp	-	The page to display the OAuth Proxy error message.

5. Securing API Gateway Access to SGFinDex Supporting APIs with OAuth 2.0

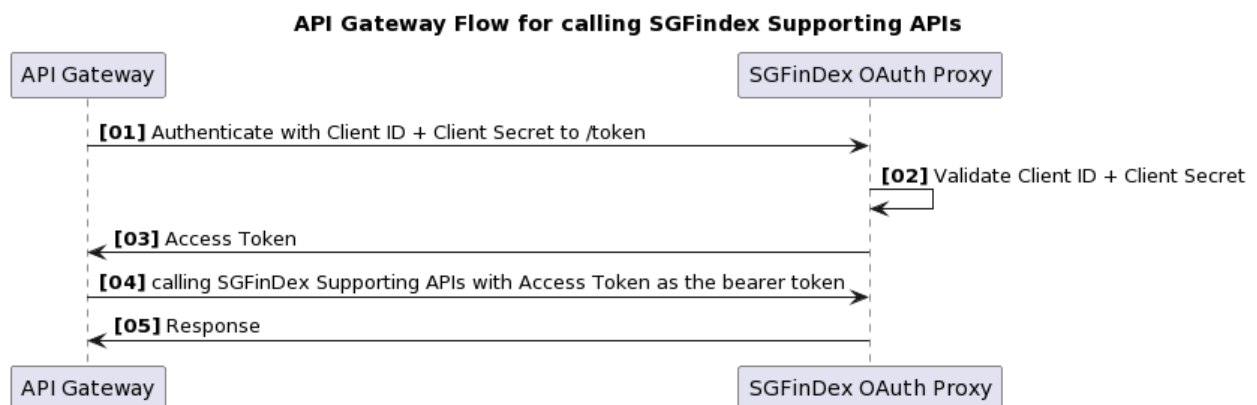
The SGFinDex OAuth Proxy supporting APIs provide the basic services to complement the role of the FISP API gateway here - to authenticate the caller of the Get Policy Details API and also to protect the returned policy detail in transit - by conforming to the standards of SGFinDex.

The request or response payload of the supporting APIs may disclose the identity and the confidential policy details of the end user. Therefore, these APIs are considered as protected resources and should be protected with authorized access.

To secure API gateway access to the SGFinDex supporting APIs, the SGFinDex OAuth Proxy requires the API gateway to authenticate to its supporting APIs using access token obtained from OAuth 2.0 Client Credential grant. This grant is commonly used for authorization between server-to-server applications.

i **Note:** You may refer to the explanation of the Client Credential flow under [UAM OAuth and OIDC Implementation Guide - Client Credentials Flow](#).

This section shows the authentication of the API Gateway to SGFinDex supporting APIs using OAuth 2.0 Client Credential grant. It illustrates the sequential flow of the API gateway to obtain the access token to be used as bearer token for calling SGFinDex OAuth Proxy Supporting APIs:



Below are the steps to implement the Client Credentials flow between the FISP API gateway and the SGFinDex OAuth Proxy:

- [Create FISP API Gateway as OAuth client with Client Credential grant in AccessMatrix Admin Console](#)
 - [Configure FISP API Gateway OAuth Client ID in SGFinDex OAuth Proxy](#)
-

-
- [Configure the FISP API Gateway to use the Client Credentials flow](#)
 - [Access the protected resources - SGFinDex Supporting APIs](#)

Create OAuth Client with Client Credential Grant for API Gateway

Perform the steps below to create the OAuth client for the FISP API gateway in the AccessMatrix Admin Console:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select **Confidential** option as the client type and click **Next >**.
3. Select **Server To Server App** option as the application type and click **Next >**.
4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
*OAuth Client Id	Unique identifier of OAuth client, e.g. "gwApp".
*OAuth Client Name	Name of OAuth client, e.g. "gwApp".
Description	Description of OAuth client.

i **Note:** Refer to [UAM OAuth and OIDC Implementation Guide - Configure Client Credentials Flow for Server To Server Application](#) for the list of attributes and its descriptions.

5. Check the **Allowed to call introspection for other OAuth Clients** checkbox to allow this API gateway to call the Introspection endpoint on behalf of other OAuth clients.
6. Click the **Finish** button. The **Create OAuth Client** page is populated with the information configured in the **OAuth Client Configuration Wizard**.
7. Usually, an access token is a short-lived token used for authorization to the protected resource. Configure **Access Token Expiry** for the validity period of the access token accordingly. Default value is 3600 seconds.
8. Save the changes.

i **Note:** Refer to [UAM OAuth and OIDC Implementation Guide - Configure Client Credentials Flow for Server To Server Application](#) for the list of attributes and its descriptions.

Configure API Gateway Client ID in SGFinDex OAuth Proxy

Perform the following steps to configure the whitelisted Gateway Client ID:

1. Navigate to `<SGF_HOME>/WEB-INF/classes/` and open **sgf.yml**.
2. Look for the `gatewayClientId` key and specify the value as shown below:

XML
<pre>supportingAPIsAuthentication: gatewayClientId: gwApp</pre>

- `gatewayClientId` - refers to the OAuth client identifier of the [API gateway](#) auto-generated in the Admin Console.
3. Save the changes.
 4. Restart the SGFinDex OAuth Proxy for changes to take effect.

Configure API Gateway to Use Client Credentials Flow

1. This setup requires the credential values of the OAuth client - the OAuth client_id and the OAuth client secret - from the OAuth client configuration in the AccessMatrix Admin Console, as mentioned [here](#).
2. In order for the FISP API gateway to access the SGFinDex supporting APIs, it needs to call the [Token endpoint](#) of the SGFinDex OAuth Proxy with the credential values to obtain the access token. The access token must be securely stored in the cache.
3. The access token is a short-lived token used for authorization to the FISP's supporting APIs. FISP API gateway needs to periodically fetch and store the access token before it gets expired. This is to prevent any disrupted access to the service consumer(s) when accessing the Get Policy Details API.

Calling the SGFinDex Supporting APIs

There are two SGFinDex supporting APIs that will require API gateway token - the access token of the API gateway's OAuth client obtained from the previous section - as its client authentication. It is set as the bearer token on the Authorization header as shown below:

<code>Authorization:Bearer <API_gateway_token></code>

For more details, please refer to the API specifications below:

- [Verify Signature Token Introspection API](#)
- [Sign and Encrypt API](#)

6. UAM SGFinDex OAuth Proxy APIs and Endpoints

The following sections provide information on the various OAuth SGFinDex-related APIs and endpoints:

- [SGFinDex OAuth Proxy Endpoints](#)
- [SGFinDex Supporting APIs](#)

6.1. SGFinDex OAuth Proxy Endpoints

Authorization Code Request Endpoint

This endpoint is to receive the authorization code request. If the user is not yet authenticated or if the user requires a reauthentication, this endpoint will redirect the end user to the custom login page, so that the user can complete the authentication/reauthentication process and also for the user to give consent.

A WSASID cookie is required as a valid user session to be provided for the authorization code request to be processed.

- To get the user authenticated, this endpoint will redirect end user to the custom login page for user authentication and consent. Click [here](#) to find out more regarding the login redirect URL setting.
- Once end user has been authenticated, the custom login page application needs to set the sessionToken (obtained from the AM5 REST API response) as the WSASID cookie. Click [here](#) to find out the important details regarding cookie setting.
- The flow will continue with user consent prompt. After user has granted consent of certain scope(s), the custom login page application needs to call [this](#) API to save user consent.

Below is the request payload and response payload of this endpoint:

HTTP Method: POST

Path: \${yourOAuthProxyURL}/authorize

This endpoint is used to obtain an authorization code.

Request Parameters

Parameter	Data Type	Description	Located	Mandatory
Signature	string	A JSON Web Token (JWT) signed using a JSON Web Signature (JWS) mechanism.	Query	Y
client_id	string	This identifier is assigned when the RP is registered with the FISP.	Query	Y

Signature JWT Payload structure fields:

Parameter	Data Type	Description	Length	Mandatory
iss	string	client_id of the signature generator. This value should match the client_id value in the request header	36	Y
iat	number	Time of issuance of request in epoch format.	10	Y
response_type	string	The client informs the authorization server which flow to execute. Default value: code	20	Y
scope	string	A space delimited list of scopes, which indicate the access to the user's data requested by the client. It MUST be a sub-set of the scopes that were registered during client registration.	256	Y
state	string	The parameter may be of any format. The value will then play-back in the redirect_uri to the client.	256	Y
redirect_uri	string	The client's callback URI during the onboarding process.	256	Y

Example of Signature JWT structure	
Header of JWT	
Code	
	<pre>{ "alg": "RS256", "typ": "JWT" }</pre>
Payload of JWT	
Code	
	<pre>{ "iss": "107b9f3e...a3350e7b8ce0",</pre>

Example of Signature JWT structure

```
{
  "iat": 1641527462,
  "response_type": "code",
  "scope": "policydetails profile",
  "state": "6pgV6032mWYRRmQ",
  "redirect_uri": "https://host:port/path"
}
```

Response Result

The response will HTTP redirect to the preregistered redirect URI of the client.

Field	Data Type	Description	Located
code	string	A temporary code that the client will exchange for an access token.	Query
state	string	Playback value from request.	Query

Possible Responses

Response Code	Response Message	HTTP Status Code	Remark
400	The request fields have missing mandatory or contain invalid parameters.	400	This error occurs when the request is not formatted correctly. For example: • one or more of the mandatory fields are missing, • parameter name or value is invalid, or • invalid client • invalid redirect URL
401	The request's authorization credentials are missing or invalid.	401	• Missing or invalid signature parameter • Signature verification failed
500	We are unable to process your request.	500	An internal server error.

Examples

Request Example (Success)
Code GET https://{OAuthProxySGFURL}/authorize?Signature=eyJraWQiOiJwa0hEdUUwQ1ljOWF6. ..7jVrIpHU39usDxtbr1eGVA&client_id=C0XiSgillI94EktbWLXWiw
Response Examples (Success)
Code https://{redirect_uri}?code=060001fRovP- WVf12mJTMeOGUWM...HuvqixZYI7Sg4y6iGydFUzq&state=6pgV6032mWYRRmQ
Response Examples (Error)
Error returned by redirection When the error occurs, the browser will be redirected to the redirect URI with error parameters: Code http://{redirect_uri}?respCode=400&respMessage=The%20request%20f ields%20have%20missing%20mandatory%20or%20contain%20invalid%20pa rameters Error returned in error page End user's browser would be forwarded to an error page (of SGFinDex OAuth Proxy) during the request validation of the authorize endpoint if either one of the following errors occurred: invalid signature missing signature invalid redirect uri missing redirect uri invalid client
Note: Read here to find out more about the configuration of the error page URL.

Get Token Endpoint

This endpoint returns access tokens, refresh tokens and scopes. The [Authorization Code flow](#), [refresh token](#), or [Client Credentials flow](#) use-cases are processed depending on the grant type in the JWT signature that is submitted in the request parameter.

Exchange Authorization Code

HTTP Method: POST

Path: \${yourOAuthProxyURL}/token

Content Type: application/x-www-form-urlencoded

This endpoint is used to exchange for access token and refresh token with an authorization code.

Request are to be post in application/x-www-form-urlencoded format.

Request Parameters

Parameter	Data Type	Description	Located	Mandatory
client_id	string	This identifier is assigned when the RP is registered with the FISP.	Header	Y
Signature	string	A JSON Web Token (JWT) signed using a JSON Web Signature (JWS) mechanism.	Header	Y

Signature JWT Payload structure fields:

Parameter	Data Type	Description	Length	Mandatory
iss	string	client_id of the signature generator. This value should match the client_id value in the request header.	36	Y
iat	number	Time of issuance of request in epoch format.	10	Y
client_secret	string	A unique identifier used to authenticate client_id. This identifier is provided when the calling application is registered with the FISP.	256	Y
grant_type	string	The client informs the authorization server of the desired grant type using this parameter. Default value: authorization_code	20	Y

Parameter	Data Type	Description	Length	Mandatory
redirect_uri	string	The client's callback URI during the onboarding process.	256	Y
code	string	The authorization code received in the redirect call of get authorization code request.	60	Y

Example of Signature JWT structure	
Header of JWT	
JSON	
<pre>{ "alg": "RS256", "typ": "JWT" }</pre>	
Payload of JWT	
JSON	
<pre>{ "iss": "107b9f3e...a3350e7b8ce0", "iat": 1641527462, "client_secret": "GBWV3sdfg...aZofCjBsP", "grant_type": "authorization_code", "redirect_uri": "https://host:port/path", "code": "060001JHxanTqC...WAAm1S4Sk38QYilHZ1-XeQ" }</pre>	

Response Result

Field	Data Type	Description	Located
access_token	string	The token that can be sent to the FISP to access the scope resources of the PSU (user). Access token should be in JWT format.	Body
refresh_token	string	A token that can be used to obtain a new access token. The refresh token may be in opaque format.	Body
scope	string	A space delimited list of scopes, which indicate the access to the user's data requested by the client.	Body

Field	Data Type	Description	Located
token_type	string	Indicates the type of token returned. This field always has a value of "Bearer".	Body
expires_in	number	The remaining lifetime on the access token (in seconds).	Body

Possible Responses

Response Code	Response Message	HTTP Status Code	Remark
400	The request fields have missing mandatory or contain invalid parameters.	400	This error occurs when the request is not formatted correctly. For example: - one or more of the mandatory fields are missing, - parameter name or value is invalid, or - invalid client - invalid grant
401	The request's authorization credentials are missing or invalid.	401	- Missing or invalid signature parameter (eg. invalid client_secret) - Signature verification failed
500	We are unable to process your request.	500	An internal server error.

Examples

Request Example (Success)	
POST request with Headers	
Code	
	<pre>Content-Type: application/x-www-form-urlencoded client_id: 107b9f3e...a3350e7b8ce0 Signature: eyJraWQiOiJwa0h...Fg1QWdaLVVBR2hzbVRJYV https://{OAuthProxySGFURL}/token</pre>
Response Examples (Success)	

Request Example (Success)	
Response in JSON	
JSON	<pre>{ "access_token": "eyJhbGciOiJSUzI...", "refresh_token": "N7rWR808iOn5V...", "scope": "address openid email", "token_type": "Bearer", "expires_in": 3599 }</pre>
Response Examples (Error)	
Response in JSON	
JSON	<pre>{ "respMessage": "The request fields have missing mandatory or contain invalid parameters.", "respTime": "30-12-2021 15:15:03.026", "respCode": "400" }</pre>

Token Refresh

HTTP Method: POST

Path: \${yourOAuthProxyURL}/token

Content Type: application/x-www-form-urlencoded

This endpoint is used to obtain new access token and refresh token using a valid refresh token.

Request Parameters

The following parameters can be posted as a part of the URL-encoded form values to the API.

Parameter	Data Type	Description	Located	Mandatory
client_id	string	This identifier is assigned when the RP is registered with the FISP.	Header	Y
Signature	string	A JSON Web Token (JWT) signed using a JSON Web Signature (JWS) mechanism.	Header	Y

Signature JWT Payload structure fields:

Parameter	Data Type	Description	Length	Mandatory
iss	string	client_id of the signature generator. This value should match the client_id value in the request header.	36	Y
iat	number	Time of issuance of request in epoch format.	10	Y
client_secret	string	A unique identifier used to authenticate client_id. This identifier is provided when the calling application is registered with the FISP.	256	Y
grant_type	string	The client informs the authorization server of the desired grant type using this parameter. Default value: refresh_token	20	Y
redirect_uri	string	The client's callback URI during the onboarding process.	256	Y
refresh_token	string	The refresh_token returned as response of get access token request.	80	Y
integrationDate	string	The current date. This is used to calculate the refresh token's expiry, with the token's lifetime starting from the integrationDate.	8	Y

Example of Signature JWT structure
Header of JWT
JSON
<pre>{ "alg": "RS256", "typ": "JWT" }</pre>
Payload of JWT

Example of Signature JWT structure

JSON

```
{
  "iss": "107b9f3e...a3350e7b8ce0",
  "iat": 1641527462,
  "client_secret": "GBWV3sdfg...aZofCjBsP",
  "grant_type": "refresh_token",
  "redirect_uri": "https://host:port/path",
  "refresh_token": "060001JHxanTqC...WAAmlS4Sk38QYilHZ1-XeQ",
  "integrationDate": "21112022"
}
```

Response Result

Field	Data Type	Description	Located
access_token	string	The token that can be sent to the FISP to access the scope resources of the PSU (user). Access token should be in JWT format.	Body
refresh_token	string	A token that can be used to obtain a new access token. The refresh token may be in opaque format.	Body
scope	string	A space delimited list of scopes, which indicate the access to the user's data requested by the client.	Body
token_type	string	Indicates the type of token returned. This field always has the value "Bearer".	Body
expires_in	number	The remaining lifetime on the access token (in seconds).	Body

Possible Responses

Response Code	Response Message	HTTP Status Code	Remark
400	The request fields have missing mandatory or contain invalid parameters.	400	This error occurs when the request is not formatted correctly. For example: one or more of the mandatory fields are missing,

Response Code	Response Message	HTTP Status Code	Remark
			- parameter name or value is invalid, or - invalid Client - invalid grant
401	The request's authorization credentials are missing or invalid.	401	- Missing or invalid Signature Parameter (eg. invalid client_secret) - Signature verification failed
500	We are unable to process your request.	500	An internal server error.

Examples

Request Example (Success)	
POST request with Headers	
Code	
	<pre>Content-Type: application/x-www-form-urlencoded client_id: 107b9f3e...a3350e7b8ce0 Signature: eyJraWQiOiJwa0h...FglQWdaLVVBR2hzbVRJYV https://{OAuthProxySGFURL}/token</pre>
Response Examples (Success)	
Response in JSON	
JSON	
	<pre>{ "access_token": "eyJhbGciOiJSUzI...", "refresh_token": "N7rWR8O8iOn5V...", "scope": "address openid email", "token_type": "Bearer", "expires_in": 3599 }</pre>
Response Examples (Error)	
Response in JSON	
JSON	

Request Example (Success)

```
{
  "respMessage": "The request fields have missing mandatory
or contain invalid parameters.",
  "respTime": "30-12-2021 15:15:03.026",
  "respCode": "400"
}
```

Client Credential

HTTP Method: POST

Path: \${yourOAuthProxyURL}/token

Content Type: application/x-www-form-urlencoded

This endpoint is used to obtain access token through the Client Credentials flow that can be used as API gateway token.

Request Parameters

The following parameters can be posted as a part of the URL-encoded form values to the API.

Parameter	Data Type	Description	Located	Mandatory
client_id	string	The client Id generated by AccessMatrix server for each registered OAuthClient.	Body	Y
client_secret	string	The client secret generated by AccessMatrix server for each registered OAuthClient.	Body	Y
grant_type	string	The client informs the authorization server of the desired grant type using this parameter. Default value: client_credentials	Body	Y

Response Result

Field	Data Type	Description	Located
access_token	string	The API gateway token that can be used as the client authentication to access the supporting APIs.	Body

Field	Data Type	Description	Located
token_type	string	Indicates the type of token returned. This field always has the value of "Bearer".	Body
expires_in	number	The remaining lifetime on the access token (in seconds).	Body

Possible Responses

HTTP Status Code	Error	Remark
400	invalid_client	The specified OAuth client_id or client_secret is invalid.
400	invalid_request	The request is missing a necessary parameter.
400	invalid_grant	OAuth client's token endpoint authentication method is invalid.
500	server_error	The server encountered an internal error.

Examples

Request Example (Success)		
POST request in parameters <table> <tr> <th>Code</th></tr> <tr> <td> Content-Type: application/x-www-form-urlencoded client_id=107b9f3e...a3350e7b8ce0& client_secret=GBWV.....BsPl& grant_type=client_credentials https://{OAuthProxySGFURL}/token </td></tr> </table>	Code	Content-Type: application/x-www-form-urlencoded client_id=107b9f3e...a3350e7b8ce0& client_secret=GBWV.....BsPl& grant_type=client_credentials https://{OAuthProxySGFURL}/token
Code		
Content-Type: application/x-www-form-urlencoded client_id=107b9f3e...a3350e7b8ce0& client_secret=GBWV.....BsPl& grant_type=client_credentials https://{OAuthProxySGFURL}/token		
Response Examples (Success)		
Response in JSON <table> <tr> <th>JSON</th></tr> <tr> <td> <pre>{ "access_token": "eyJhbGciOiJSUzI...", "token_type": "Bearer", "expires_in": 3599 }</pre> </td></tr> </table>	JSON	<pre>{ "access_token": "eyJhbGciOiJSUzI...", "token_type": "Bearer", "expires_in": 3599 }</pre>
JSON		
<pre>{ "access_token": "eyJhbGciOiJSUzI...", "token_type": "Bearer", "expires_in": 3599 }</pre>		
Response Examples (Error)		

Request Example (Success)
Response in JSON
JSON
<pre>{ "error": "invalid_client", "error_description": "Invalid client id or secret" }</pre>

Revoke Token Endpoint

This endpoint is used for revocation of the [refresh token](#) and [API gateway token](#) (the access token obtained through the Client Credentials flow).

Revoke Refresh Token

HTTP Method: POST

Path: \${yourOAuthProxyURL}/revoke

Content Type: application/x-www-form-urlencoded

This endpoint is used to revoke refresh token.

Request Parameters

The following parameters can be posted as a part of the URL-encoded form values to the API.

Parameter	Data Type	Description	Located	Mandatory
client_id	string	This identifier is assigned when the RP is registered with the FISP.	Header	Y
Signature	string	A JSON Web Token (JWT) signed using a JSON Web Signature (JWS) mechanism.	Header	Y

Signature JWT Payload structure fields:

Parameter	Data Type	Description	Length	Mandatory
iss	string	client_id of the signature generator. This value should match the client_id value in the request header.	36	Y
iat	number	Time of issuance of request in epoch format.	10	Y
client_secret	string	A unique identifier used to authenticate client_id. This identifier is provided when the calling application is registered with the FISP.	256	Y
token	string	The refresh token to be revoked.	-	Y
token_type_hint	string	Type of token. Default value: refresh_token	20	N

Example of Signature JWT structure
Header of JWT JSON <pre>{ "alg": "RS256", "typ": "JWT" }</pre>
Payload of JWT JSON <pre>{ "iss": "107b9f3e...a3350e7b8ce0", "iat": 1641527462, "client_secret": "GBWV3sdfg...aZofCjBsP", "token": "060001JHxanTqC...WAAmLS4Sk38QYilHZl-XeQ", "token_type_hint": "refresh_token" }</pre>

Response Result

Field	Data Type	Description	Located
respCode	string	Response code.	Body(JSON)

Field	Data Type	Description	Located
respMessage	string	Response description.	Body(JSON)
respTime	Date	Response time.	Body(JSON)

Possible Responses

Response Code	Response Message	HTTP Status Code	Remark
0000	Successful	200	A successful revocation is denoted by an HTTP 200 OK response.
400	The request fields have missing mandatory or contain invalid parameters.	400	This error occurs when the request is not formatted correctly. For example: - one or more of the mandatory fields are missing, - parameter name or value is invalid, or - invalid client
401	The request's authorization credentials are missing or invalid.	401	- Missing or invalid Signature Parameter (eg. invalid client_secret) - Signature verification failed
500	We are unable to process your request.	500	An internal server error.

Examples

Request Example (Success)	
POST request with Headers	
Code	
<pre>Content-Type: application/x-www-form-urlencoded client_id: 107b9f3e...a3350e7b8ce0</pre>	

Request Example (Success)
Signature: eyJraWQiOiJwa0h...Fg1QWdaLVVBR2hzbVRJYV https://{OAuthProxySGFURL}/revoke
Response Examples (Success)
Response in JSON <pre>JSON { "respMessage": "Successful", "respTime": "30-12-2021 15:15:03.026", "respCode": "0000" }</pre>
Response Examples (Error)
Response in JSON <pre>JSON { "respMessage": "The request fields have missing mandatory or contain invalid parameters.", "respTime": "30-12-2021 15:15:03.026", "respCode": "400" }</pre>

Revoke Gateway Token

HTTP Method: POST

Path: \${yourOAuthProxyURL}/revoke

Content Type: application/x-www-form-urlencoded

This endpoint is used to revoke the API gateway token, an access token obtained through the Client Credentials flow.

Request Parameters

The following parameters can be posted as a part of the URL-encoded form values to the API.

The call to the /revoke endpoint requires authentication. In this case, it is the [basic authentication](#) of the client ID and secret. You can find the client ID and secret in the basic information section of the **View OAuth Client** page in the Admin Console.

Parameter	Data Type	Description	Located	Mandatory
client_id	string	The client Id generated by AccessMatrix server for each registered OAuthClient.	Body	Y
client_secret	string	The client secret generated by AccessMatrix server for each registered OAuthClient.	Body	Y
token	string	The access token to be revoked.	Body	Y
token_type_hint	string	Type of token. Default value: access_token	Body	N

Response Result

A successful revocation is denoted by an HTTP 200 OK response with no content in the response body.

Possible Responses

HTTP Status Code	Error	Remark
400	invalid_request	The request is missing a necessary parameter.
401	invalid_client	The specified OAuth client_id or client_secret is invalid.
500	server_error	The server encountered an internal error.

Examples

Request Example (Success)
POST request in parameters
Code <pre>Content-Type: application/x-www-form-urlencoded client_id=107b9f3e...a3350e7b8ce0& client_secret=GBWV.....BsPl& token=eyJhbGciOiJSUzI...& token_type_hint=access_token https://{OAuthProxySGFURL}/revoke</pre>
Response Examples (Success)

Request Example (Success)	
Response in JSON with empty body	
JSON	
{ }	
Response Examples (Error)	
Response in JSON	
JSON	
{	
"error": "invalid_client",	
"error_description": "Invalid client id or secret"	
}	

JWKS Endpoint

This endpoint is used to publish the OP signing public Key for token signature verification.

HTTP Method: GET

Path: \${yourOAuthProxyURL}/jwks

This endpoint is used to publish the OP signing public Key.

Response Result

Field	Data Type	Description
keys	Array	Array of the Java Web Keys.
kti	string	The family of cryptographic algorithms used with the key.
kid	string	The unique identifier for the key.
use	string	How the key was meant to be used. sig : for signing. enc: for encryption.
alg	string	The specific cryptographic algorithm used with the key.
n	string	The modulus for the RSA public key.
e	string	The exponent for the RSA public key.

Field	Data Type	Description
x5c	Array	The x.509 certificate chain. The first entry in the array is the certificate to use for token verification; the other certificates can be used to verify this first certificate.

Examples

Request Example (Success)	
Code	
	GET https://{OAuthProxySGFURL}/jwks
Response Examples (Success)	
Code	
	<pre>{ "keys": [{ "kty": "RSA", "kid": "pkHDuE0BYc9az7KGpX5AgZ-UAGhsmTIaZJofs8gnc_A", "use": "sig", "alg": "RS256", "n": "mEJJp...9kw", "e": "AQAB", "x5c": ["MIIDg...z2twA=="] }] }</pre>

6.2. SGFinDex Supporting APIs

Verify Request API

HTTP Method: POST

Path: \${yourOAuthProxyURL}/verifyrequest

Content Type: application/x-www-form-urlencoded

This endpoint is used to verify the request of the Relying Party (RP), by verifying the JSON Web Token (JWT) Signature of the request. If the signature is valid, the JWT payload of the given JWT Signature is returned.

Request Parameters

Parameter	Data Type	Description	Located	Mandatory
Signature	string	A JSON Web Token (JWT) signed using a JSON Web Signature (JWS) mechanism.	Body	Y
client_id	string	This identifier is assigned when the RP is registered with the FISP.	Body	Y

Sample Signature JWT Payload structure fields (e.g. from SGFinDex OAuth /authorize Endpoint):

Parameter	Data Type	Description	Length	Mandatory
iss	string	client_id of the signature generator. This value should match the client_id value in the Request parameter.	36	Y
iat	number	Time of issuance of request in epoch format.	10	Y
response_type	string	The client informs the authorization server (FISP) which flow to execute	20	Y
scope	string	A space delimited list of scopes, which indicate the access to the user's data requested by the client. The custom login page must prompt the user for consent based on the scope(s)	256	Y

Parameter	Data Type	Description	Length	Mandatory
		specified here. Value: productBalances		
redirect_uri	string	The client's callback URI where the authentication response (authorization code) will be sent.	256	Y
state	string	The parameter may be of any format and is opaque to the FISP. This value is used as password to call login for SGFinDex Identity cross-checking.	256	Y
integrationDate	Date	The date of the first link up account. The refresh token will be valid for 365 days starting this date. Format: DDMMYYYY	8	Y

Example of Signature JWT structure (e.g. from SGFinDex /authorize Endpoint)
Header of JWT JSON <pre>{ "alg": "RS256", "typ": "JWT" }</pre>
Payload of JWT JSON <pre>{ "iss": "107b9f3e-7399-4983-8342-a3350e7b8ce0", "iat": 1561548933, "response_type": "code", "scope": "policydetails", "redirect_uri": "https://host:port/path", "state": "af0ifjsldkj", "integrationDate": "08042020" }</pre>

Response Result

Field	Data Type	Description	Located
respCode	string	Response code.	Body (JSON)
respMessage	string	Response description.	Body (JSON)
respTime	Date	Response time.	Body (JSON)
signature-payload	JSONObject	JWT payload in Signature.	Body (JSON)

Possible Responses

Response Code	Response Message	HTTP Status Code	Remark
0000	Successful	200	The successful response code and message when the JWT of signature is verified.
400	The request fields have missing mandatory or contain invalid parameters.	400	This error occurs when the request is missing client id
401	The request's authorization credentials are missing or invalid.	401	This error occurs when: • invalid signature JWT • invalid Signature or missing signature • invalid client during signature verification
500	We are unable to process your request.	500	An internal server error.

Examples

Request Example (Success)	
POST request in parameters	
Code	
Content-Type: application/x-www-form-urlencoded "client_id"="107b9f3e...a3350e7b8ce0"	

Request Example (Success)
<pre>"Signature"="ceggdghe....geeg" https://{OAuthProxySGFURL}/verifyrequest</pre>
Response Examples (Success)
Response JSON <pre>{ "respMessage": "Successful", "respTime": "17-12-2021 10:05:09.767", "respCode": "0000", "signature-payload": { "iss": "107b9f3e-7399-4983-8342-a3350e7b8ce0", "iat": 1561548933, "response_type": "code", "scope": "policydetails", "redirect_uri": "https://host:port/path", "state": "af0ifjsldkj", "integrationDate": "08042020" } }</pre>
Response Examples (Error)
HTTP 400 Bad Request JSON <pre>{ "respMessage": "The request fields have missing mandatory or contain invalid parameters.", "respTime": "17-12-2021 09:31:40.621", "respCode": "400" }</pre>

Verify Signature Token Introspection API

HTTP Method: POST

Path: \${yourOAuthProxyURL}/sgfverify

Content Type: application/json

This endpoint is used to verify the JSON Web Token (JWT) of the signature and introspect the access token passing in from the Relying Party (RP).

For SGFinDex V2 API: In addition to the existing handling, the f-app-request JWT signature is validated accordingly. The signature verification process involves the retrieval of the JWK (or public key) from the

[configured f-app entity's jwksUrl](#) - if this is not found, the runtime JKU provided is invoked. Once the signature of the f-app-request is validated, the encryptionKid is returned. This value is required to be used for the subsequent Sign and Encrypt API call.

Request Parameters

Parameter	Data Type	Description	Located	Mandatory
Signature	string	A JSON Web Token (JWT) signed using a JSON Web Signature (JWS) mechanism.	Header	Y
client_id	string	This identifier is assigned when the RP is registered with the FISP.	Header	Y
token	string	RP's access token.	Body (JSON)	Y
Authorization: Bearer <API_gateway_token>	string	The API gateway token used to introspect access token for the RP.	Header	Y

Signature JWT Payload structure fields:

	Parameter	Data Type	Description	Length	Mandatory
	iss	string	The principal that issues the JWT.	36	Y
	iat	number	Time of issuance of the request in epoch format.	10	Y
	aud	string	Recipients that the JWT is intended for. The client_id of the client will used for this field.	256	Y
	Interaction-ID	string	Correlation ID of the request.	40	Y
	f-app-entity	string	FI name (static value decided by Central Infra).	60	Y
	f-app-request	string	JWT Request Signature from SC.	4000	Y
V2	Example of Signature JWT structure				
	Header of JWT				
	JSON				
	<pre>{ "alg": "RS256", "typ": "JWT" }</pre>				
V1	Payload of JWT				
	JSON				
	<pre>{ "iss": "107b9f3e...a3350e7b8ce0", "iat": 1641527462, "aud": "C0XiSgillI94EktbWLXWiw", "Interaction-ID": "YCB6nEQqviQ0SH2e", "f-app-entity": "Example PTE. LTD.", "f-app-request": "eyJraWQIOiJwa0h...Dg1QWdbLVVBR2hzbVRJYV" }</pre>				
	Parameter	Data Type	Description	Length	Mandatory
	iss	string	client_id of the signature generator. This value should match the client_id value in the Request Header	36	Y
	iat	number	Time of issuance of request in epoch format.	10	Y

	Parameter	Data Type	Description	Length	Mandatory
	iss	string	The principal that issues the JWT.	36	Y
	iat	number	Time of issuance of the request in epoch format.	10	Y
	aud	string	Recipients that the JWT is intended for. The client_id of the client will used for this field.	256	Y
	Interaction-ID	string	Correlation ID of the request.	40	Y
	f-app-entity	string	FI name (static value decided by Central Infra).	60	Y
	f-app-request	string	JWT Request Signature from SC.	4000	Y
V2	Example of Signature JWT structure				
	Header of JWT				
	JSON				
	<pre>{ "alg": "RS256", "typ": "JWT" }</pre>				
Resp	Payload of JWT				
	JSON				
	<pre>{ "iss": "107b9f3e...a3350e7b8ce0", "iat": 1641527462, "aud": "C0XiSgillI94EktbWLXWiw", "Interaction-ID": "YCB6nEQqviQ0SH2e", }</pre>				
	Field	Data Type	Description	Located	
	respCode	string	Response code.	Body (JSON)	
	respMessage	string	Response description.	Body (JSON)	
	respTime	Date	Response time.	Body (JSON)	

Field	Data Type	Description	Located
signature-payload	JSONObject	JWT payload in the signature.	Body (JSON)
token-payload	JSONObject	Access token payload.	Body (JSON)
f-app-entity	String	FI Name (static value decided by Central Infra). Note: Only applicable to v2.	Body (JSON)
encryptionKid	String	KID for the key to be used for encryption (the key Id to be fetched from the JWKS endpoint of the SC for encryption purposes). Note: Only applicable to v2.	Body (JSON)

Token Payload structure fields:

Field	Data Type	Description
active	Boolean	Indicate whether or not the presented token is currently active.
scope	String	Contains a space-separated list of scopes associated with this token.
client_id	String	Client identifier for the RP that requested this token.
username	String	Human-readable identifier for the resource owner who authorized this token.
token_type	String	Type of the token, either access_token or refresh_token.
exp	Integer	Integer timestamp, measured in the number of seconds since January 1 1970 UTC, indicating when this token will expire.
iat	Integer	Time of issuance of the token was originally issued in epoch format.
jti	String	Unique identifier for the token.
sub	String	Subject of the token. Usually a machine-readable identifier of the resource owner who authorized this token. Name resolved from OAuth client Value to return as Subject (sub) or User Response to return as Subject (sub) value attribute.
aud	String	Service-specific string identifier or list of string identifiers representing the intended audience for this token. This value is configured with OAuth Client Id authorized by resource owner for now.
iss	String	Issuer of the token.

Signature Payload structure fields:

V2	Field	Data Type	Description
	Interaction-ID	String	Correlation ID of Request.
	f-app-entity	String	FI name (static value decided by Central Infra).
	f-app-request	String	JWT Request Signature from SC.
	f-app-request JOSE Header fields:		
	Field	Data Type	Description
	typ	String	Token type; JWT.
	alg	String	Cryptographic algorithm used to secure the JWS.
	jku	String	A resource for a set of JSON-encoded public keys, one of which corresponds to the key used to digitally sign the JWS.
	kid	String	Indicates which key was used to secure the JWS.
	f-app-request JOSE Payload fields:		
	Field	Data Type	Description
	consentId	String	The unique identifier to recognize consent, uniquely recorded at the SC for the SP to extend original consent to T+364 days. Note: Consent Id needs to be same for both the correlated token and data_pull calls.
	encryptionKid	String	The KID for the key to be used for encryption (the Key Id to be fetched from the JWKS endpoint of the SC for encryption purposes).
V1	Field	Data Type	Description
	Interaction-ID	String	Correlation ID of Request.
	f-app-id	String	Client id of front-end/service consumer app.
	f-app-name	String	Short name of front-end/service consumer app.

Possible Responses

Response Code	Response Message	HTTP Status Code	Remark
0000	Successful	200	The successful response code and message when the JWT of signature is verified and access token is introspected.
400	The request fields have missing mandatory or contain invalid parameters.	400	This error occurs when the request is missing client id
401	The request's authorization credentials are missing or invalid.	401	This error occurs when: - the request does not provide token to be introspected or the access token provided is invalid (failed token introspection) - invalid signature JWT - invalid signature or missing signature - invalid bearer token or the bearer token does not have permission to introspect 3rd-party tokens. - invalid client during signature verification or access token introspection
500	We are unable to process your request.	500	An internal server error.

Examples

Request Example (Success)	
Authentication using bearer API gateway token and client_id in the Authorization header	
Code	<pre>POST \${yourOAuthProxyURL}/sgfverify Authorization: Bearer \${API_gateway_token} client_id: 107b9f3e...a3350e7b8ce0 Signature: eyJraWQiOiJwa0h...Fg1QWdaLVVBR2hzbVRJYV {"token"="eyJhbGciOiJSU0ExXzUiLA0...faENudldoNkllWUkydz"}</pre>
Response Examples (Success)	

Request Example (Success)	
Response	
JSON	<pre>{ "respMessage": "Successful", "respTime": "17-12-2021 10:05:09.767", "respCode": "0000", "token-payload": { "active": true, "scope": "policydetails profile", "client_id": "C0XiSgillI94EktbWLXWiw", "username": "demouser", "token_type": "access_token", "exp": 1642070737, "iat": 1642067137, "jti": "060001REAjheG-tPqG7qtmDqStX6Dne...lJ0IFFcvJfI40hWaaZVMdHtPGx", "sub": "S2203993", "aud": "C0XiSgillI94EktbWLXWiw", "iss": "i-sprint.com"}, "signature-payload": { "Interaction-ID": "YCB6nEQqviQ0SH2e", "f-app-id": "SCA-140749", "f-app-name": "ExcellentCompanyApp"} }</pre>
Response Examples (Error)	
HTTP 400 Bad Request	
JSON	<pre>{ "respMessage": "The request fields have missing mandatory or contain invalid parameters.", "respTime": "17-12-2021 09:31:40.621", "respCode": "400" }</pre>

Sign and Encrypt API

HTTP Method: POST

Path: \${yourOAuthProxyURL}/signencrypt

Content Type: application/json

This endpoint is used to sign and encrypt user's policy details and return as a JSON Web Encryption (JWE).

Request Parameters

Parameter	Data Type	Description	Located	Mandatory
data	string	User policy details.	Body(JSON)	Y
encryptionCertificate	string	The certificate to be used for encryption is obtained from the encryptionCertificate parameter of the request body when the Policy Details API is called. Only applicable to v1.	Body(JSON)	Y
f-app-entity	string	The f-app-entity obtained from sgVerify response payload. Only applicable to v2.	Body(JSON)	Y
encryptionKid	string	The encryptionKid obtained from sgVerify response payload. Only applicable to v2.	Body(JSON)	Y
Authorization:Bearer <API_gateway_token>	string	The API gateway token is used as the client authentication in order to use this API.	Header	Y

Response Result

Field	Data Type	Description	Located
seData	string	User policy details signed and encrypted in JWE payload. This field will only will be returned if the status code is 200.	Body(JSON)
respCode	string	Response code.	Body(JSON)
respMessage	string	Response description.	Body(JSON)
respTime	Date	Response time.	Body(JSON)

Possible Responses

Response Code	Response Message	HTTP Status Code	Remark
0000	Successful	200	The successful response code and message when the payload is successfully signed and encrypted.
400	The request fields have missing mandatory or contain invalid parameters.	400	This error occurs when the request is not formatted correctly. For example: - one or more of the mandatory fields are missing, or - parameter name or value is invalid
401	The request's authorization credentials are missing or invalid.	401	This error occurs when the API gateway token is not found, invalid or expired.
403	Forbidden.	403	This error occurs when there is an issue with the passed encryption certificate.
404	The resource is empty resource or the resource cannot be found.	404	This error occurs when the data is missing or blank.
500	We are unable to process your request.	500	An internal server error.

Examples

Request Example (Success)	
Authentication using bearer API gateway token in the Authorization header	
Code	<pre>POST \${yourOAuthProxyURL}/signencrypt Host: {OAuthProxySGFURL}.com Content-Length: 1182 Content-Type: application/json Authorization: Bearer \${API_gateway_token} { "data": "{ "timestamp": "2020-05-13T09:49:47.602Z", "fispName": "ABC Insurance Pte Ltd", "fisp": "I230D", "respCode": "0000",</pre>

Request Example (Success)

```
"respMessage": "The request fields have missing mandatory or  
contain invalid parameters.",  
"asOfMonthEndDate": "2021-07-31",  
"policies": [  
  {  
    "productClass": "Participating",  
    "productType": "WholeLife",  
    "policyNo": "12345678",  
    "surrenderValue": "550000.00",  
    "surrenderValuedate": "2021-07-31",  
    "nav": "40000.00",  
    "navDate": "2021-07-25",  
  }]  
}],  
  
"encryptionCertificate": "MIIDQjCCAIqgAwIBAgIGA...dS4xSvdXK3IVfOWA=="  
}
```

Response Examples (Success)

Response

JSON

```
{  
  "respMessage": "Successful",  
  "respTime": "17-12-2021 10:05:09.767",  
  "respCode": "0000",  
  
  "seData": "eyJhbGciOiJSU0EtT0FFUC0yNTYiLCJlbmMiOiJBMjU2Q0JDLUhTNTEyIiwia3R5IjoiaSldUin0.AtVvmz1CRsfH7G  
    ...<truncated>...-WbUC4Ywd9rWc8eg.xGYWgF7lBe6BLU412ilFmA.NeWHkUfrO3gaQ-  
    zZFEUJ_yD3hYH2-  
    G29P6fzRp7sj5E.b2nDYBf8CCgaYgibSexaYBExCE6a1QbXhWxNY5HPEBY"  
}
```

Response Examples (Error)

HTTP 400 Bad Request

JSON

```
{  
  "respMessage": "Forbidden.",  
  "respTime": "17-12-2021 09:31:40.621",  
  "respCode": "403"  
}
```

7. Appendices

A. Basic Authentication

Basic authentication is a simple authentication scheme built into the HTTP protocol. The client sends HTTP requests with the Authorization header that contains the word Basic, followed by a space and a base64-encoded string username:password, i.e. <client_id>:<client_secret>.
